دانشگاه صنعتی امیرکبیر
(پلی‌تکنیک تهران)
دانشکده مهندسی کامپیوتر

پایان‌نامه کارشناسی
مهندسی کامپیوتر

طراحی و پیاده‌سازی سامانه مدیریت تعاونی‌های مسکن

نگارش
آرمینا صادقی مجرد

استاد راهنما
دکتر مسعود صبایی

آذر ۱۴۰۲

به نام خدا

تاریخ: ۱۴۰۲/۰۹/۲۹

تعهدنامه اصالت اثر

اینجانب آرمینا صادقی مجرد متعهد می‌شوم که مطالب مندرج در این پایان نامه حاصل کار پژوهشی اینجانب تحت نظارت و راهنمایی اساتید دانشگاه صنعتی امیرکبیر بوده و به دستاوردهای دیگران که در این پژوهش از آنها استفاده شده‌است مطابق مقررات و روال متعارف ارجاع و در فهرست منابع و مآخذ ذکر گردیده است. این پایان نامه قبلاً برای احراز هیچ مدرک هم‌سطح یا بالاتر ارائه نگردیده است.

در صورت اثبات تخلف در هر زمان، مدرک تحصیلی صادر شده توسط دانشگاه از درجه اعتبار ساقط بوده و دانشگاه حق پیگیری قانونی خواهد داشت.

کلیه نتایج و حقوق حاصل از این پایان نامه متعلق به دانشگاه صنعتی امیرکبیر می‌باشد. هرگونه استفاده از نتایج علمی و عملی، واگذاری اطلاعات به دیگران یا چاپ و تکثیر، نسخه‌برداری، ترجمه و اقتباس از این پایان نامه بدون موافقت کتبی دانشگاه صنعتی امیرکبیر ممنوع است. نقل مطالب با ذکر مآخذ بلامانع است.

آرمینا صادقی مجرد

امضا

تقدیم

تقدیم به پدر و مادر عزیزم که همیشه حامی و پشتیبان من بودند و محیطی سرشار از امنیت و آرامش را برای من فراهم نمودند تا بتوانم در راستای علایق خویش قدم بردارم.

سپاس‌گزاری

نمی‌توانم معنایی بالاتر از تقدیر و تشکر بر زبانم جاری سازم و سپاس خود را در وصف استاد خویش جناب آقای دکتر مسعود صبایی آشکار نمایم که پیمودن این مسیر بدون توصیه‌های علمی دقیق ایشان ناممکن بود. همچنین از استاد گران‌مایه، جناب آقای دکتر سیاوش خرسندی که زحمت داوری این پایان‌نامه را برعهده داشتند، نهایت تشکر را دارم.

آرمینا صادقی مجرد

آذر ۱۴۰۲

چکیده

تعاونی‌های مسکن سازمان‌هایی هستند که در حوزه مسکن فعالیت می‌کنند. این تعاونی‌ها به منظور ارائه‌ی خدمات مسکن به اعضاء و تأمین مسکن متقاضیان و همچنین مدیریت مالی، نیاز به سامانه‌های مدیریتی مدرن دارند. این سامانه‌ها باید توانایی مدیریت تعداد کاربران زیاد را دارا باشد. همچنین با توجه به گردش مالی قابل توجه، سیستم باید از امنیت بالایی در حوزه‌ی صحت داده‌ها[1] و درستی محاسبات برخوردار باشد.

در این پروژه به طراحی و پیاده‌سازی یک سامانه‌ی الکترونیکی جهت مدیریت تعاونی‌های مسکن پرداخته شده‌است. پس از بررسی سامانه‌های موجود در این زمینه، مشکلاتی از قبیل محدودیت عضویت برای همه‌ی افراد، ضعف در واسط کاربری[2]، کمبود شفافیت و توضیحات کافی درمورد روند توسعه‌ی پروژه‌های ساختمانی و هزینه‌ی بالا مشاهده شد. در این سامانه الکترونیکی امکانات جامعی ارائه شده‌است که این امکان را فراهم می‌کند تا با کم‌ترین دخالت نیروی انسانی، بتوان تعاونی‌های مسکن ارگان‌های مختلف را مدیریت کرد. از جمله این ویژگی‌ها ثبت نام و احراز هویت[3] اعضاء، مدیریت اقساط اعضاء، ارتباط مؤثر با اعضاء از طریق پلتفرم الکترونیکی، و امکان نظارت بهتر بر فعالیت‌های تعاونی‌ها به وسیله‌ی گزارش‌ها و آمارهای دقیق می‌باشد.

واژه‌های کلیدی:

سامانه‌ی الکترونیکی، مدیریت تعاونی مسکن، توسعه‌ی وب، فرآیند

[1] Date Accuracy

[2] User Interface

[3] Authentication

فهرست مطالب

صفحه

فصل اول مقدمه ۲
۱-۱- نیاز موضوع ۳
۱-۲- تعریف و اهمیت مسئله ۳
۱-۳- انگیزه‌ی انجام کار ۳
۱-۴- روش انجام کار ۴
۱-۵- ساختار گزارش ۴
فصل دوم معرفی مفاهیم پایه و ابزارهای مورد استفاده در پروژه ۵
۲-۱- مفاهیم پایه ۶
۲-۱-۱- فرانت‌اند ۶
۲-۱-۲- بک‌اند ۶
۲-۱-۳- طراحی دامنه محور ۷
۲-۱-۴- معماری مبتنی بر سرویس ۷
۲-۲- معرفی ابزارها و فناوری‌های استفاده شده در پروژه ۸
۲-۲-۱- انگولار ۸
۲-۲-۲- دات‌نت ۹
۲-۲-۳- اس کیو ال سرور ۹
فصل سوم جمع آوری نیازمندی‌ها و طراحی سامانه ۱۰
۳-۱- مقدمه ۱۱
۳-۲- نیازمندی‌ها ۱۱
۳-۲-۱- نیازمندی‌های کارکردی ۱۱
۳-۲-۲- نیازمندی‌های غیرکارکردی ۱۳
۳-۳- انتخاب فناوری ۱۴
۳-۳-۱- ملاک‌ها ۱۵
۳-۳-۲- مقایسه و انتخاب فناوری ۱۵
۳-۳-۳- فرانت‌اند ۱۶
۳-۳-۴- بک‌اند ۱۶
۳-۴- طراحی ۱۶
۳-۴-۱- طراحی واسط کاربری ۱۶
۳-۴-۲- طراحی فرانت‌اند ۱۹
۳-۴-۳- طراحی بک‌اند ۲۲
۳-۵- جمع بندی ۳۰

فصل چهارم پیاده‌سازی ۳۱
۴-۱- مقدمه ۳۲
۴-۲- پیاده‌سازی فرانت‌اند ۳۲
۴-۲-۱- پیاده‌سازی فرانت‌اند ۳۳
۴-۲-۲- کتابخانه‌ی واسط کاربری ۳۴
۴-۲-۳- کتابخانه‌ی مدیریت وضعیت ۳۴
۴-۲-۴- پیاده‌سازی بک‌اند ۳۸
۴-۲-۵- نقاط انتهایی ۴۱
۴-۳- تست نیازمندی‌ها ۴۳
۴-۳-۱- نیازمندی‌های کارکردی ۴۳
۴-۳-۲- نیازمندی‌های غیرکارکردی ۴۴
۴-۴- جمع بندی ۴۸
فصل پنجم جمع‌بندی و پیشنهادات ۴۹
۵-۱- خلاصه ۵۰
۵-۲- کاربرد ۵۰
۵-۳- پیشنهادات ۵۰

فهرست اشکال

صفحه

شکل ۱- صفحه فرود پنل کاربری ۱۷
شکل ۲ - صفحه فرود پنل مدیریت ۱۸
شکل ۳ - عملکرد کلی مؤلفه احراز هویت کاربر ۲۰
شکل ۴ - عملکرد کلی مؤلفه نگه دارنده اطلاعات کاربر ۲۰
شکل ۵ - عملکرد کلی مؤلفه‌های ایجاد پروژه ۲۱
شکل ۶ - عملکرد کلی نمایش لیست همه‌ی پروژهها ۲۱
شکل ۷ - عملکرد کلی مؤلفه‌ی ایجاد قرارداد ۲۲
شکل ۸ - عملکرد کلی نمایش لیست پرداختهای یک کاربر ۲۲
شکل ۹ - نحوه‌ی کارکرد بک‌اند[۳] ۲۳
شکل ۱۰ - طراحی جدول کاربر ۲۴
شکل ۱۱ - طراحی جدول پروژه ۲۵
شکل ۱۲ - طراحی جدول قطعه ۲۶
شکل ۱۳ - طراحی جدول فاز ۲۷
شکل ۱۴ - طراحی جدول قرارداد ۲۸
شکل ۱۵ - طراحی جدول قسط قرارداد ۲۹
شکل ۱۶ – نحوه‌ی کار الگوی تزریق وابستگی در انگولار[۴] ۳۳
شکل ۱۷ - چرخه‌ی ناوبری انگولار[۵] ۳۴
شکل ۱۸ - الگوی ریداکس[۶] ۳۶
شکل ۱۹ - کارکرد اچ‌تی‌تی‌پی کلاینت[۷] ۳۸
شکل ۲۰ - کارکرد میان‌افزارها[۸] ۳۹
شکل ۲۱ - رویکرد کدابتدا برای جدول پروژه ۴۰
شکل ۲۴ – ورود به سامانه با رمز یکبار مصرف ۴۴

فهرست جداول

صفحه

جدول ۱ - نیازمندی‌های کارکردی ۱۲
جدول ۲ - نیازمندی‌های غیرکارکردی ۱۳
جدول ۳ - جزییات نقاط انتهایی ۴۱

فصل اول

مقدمه

۱-۱- نیاز موضوع

در دهه‌های اخیر با توسعه‌ی فناوری اطلاعات و ارتباطات، نیاز به بهبود و ارتقاء سیستم‌ها و فرآیندهای مختلف در حوزه‌های مختلف زندگی اجتماعی احساس می‌شود. یکی از حوزه‌های مهم و حیاتی که نیاز به بهبود فرآیندهای مدیریتی خود دارد، حوزه‌ی تعاونی‌های مسکن می‌باشد. ایجاد یک سامانه‌ی الکترونیکی جامع جهت مدیریت این تعاونی‌ها، می‌تواند در ارائه‌ی خدمات و بهره‌وری این سازمان‌ها به طور مؤثری بهبود ایجاد کند.

۱-۲- تعریف و اهمیت مسئله

تعاونی‌های مسکن به عنوان نهادهای حیاتی در ساختار اجتماعی، مسکن مورد نیاز افراد را فراهم می‌کنند. با افزایش تعداد این تعاونی‌ها، نیاز به ساماندهی و بهینه‌سازی فرآیندهای مدیریتی این سازمان‌ها افزایش یافته است. این موضوع باعث مطرح شدن چالش‌ها و مشکلاتی در زمینه‌هایی از جمله مدیریت مالی، ثبت نام[۱] اعضاء و ارتباطات داخلی گردیده است. لذا، پیاده‌سازی یک سامانه‌ی الکترونیکی جهت بهبود مدیریت تعاونی‌های مسکن می‌تواند گام مؤثری در جهت حل این مسائل محسوب شود.

۱-۳- انگیزه‌ی انجام کار

با مرور سامانه‌های مشابه، مشکلات متداولی از جمله نداشتن واسط کاربری مناسب و پیچیدگی بیش از حد در سامانه، سختی عضویت در این سامانه‌ها و زیاد بودن هزینه استفاده از این سامانه‌ها روبرو شدیم. به منظور حل این مشکلات و بهبود عملکرد فرآیندها، به طراحی سامانه‌ای پرداختیم که ویژگی‌های اساسی‌ای از جمله تجربه‌ی کاربری یکنواخت در تمام صفحات، سادگی استفاده، امکان عضویت آسان، حفظ صحت داده و کارآیی بهتر را به ارمغان می‌آورد. وجود این سامانه‌ی الکترونیکی جهت افزایش بهره‌وری و کاهش هزینه‌ها در سازمان‌های تعاونی مسکن، از اهمیت به سزایی برخوردار است.

[۱] Registration

۱-۴- روش انجام کار

سامانه‌ی پیاده‌سازی شده در این پروژه، تمامی ویژگی‌های ذکر شده برای یک سامانه مدیریت تعاونی مسکن مناسب و جامع که در انتهای بخش قبل ذکر شد را فراهم می‌کند. پروژه‌های ساختمانی توسط یک پنل مدیریت[۱] به همراه واحدهای طراحی شده برای ساختمان ایجاد می‌شوند. سپس اعضاء در پنل کاربری[۲]با مشاهده‌ی پروژه‌های موجود و شرایط و واحدهای موجود در هر یک، می‌توانند واحدهای مورد نظر خود را انتخاب کرده و با بستن قرارداد در ساخت پروژه مشارکت کنند. هر کاربر با توجه به قراردادها و اقساطی که دارد، موظف است که در هر ماه مبلغی را پرداخت کند و در نهایت با اتمام پروژه و پرداخت تمامی اقساط، مالک واحد خود شود. این سامانه از دو بخش اصلی **پنل کاربری** و **پنل مدیریت** تشکیل می‌شود. هر پنل از یک واسط کاربری و یک سرور[۳]شامل چند میکروسرویس[۴] تشکیل می‌شود. در سرور تعدادی نقطه انتهایی[۵] پیاده‌سازی می‌شوند که وظیفه‌ی آن‌ها ارسال اطلاعات سرور به فرانت‌اند[۶]و دریافت اطلاعات کاربران، پروژه‌ها و پرداخت‌ها از آن می‌باشد.

۱-۵- ساختار گزارش

در این گزارش ابتدا تعریفی از مسئله و نیاز موضوع بیان شد و سپس ضرورت و اهمیت سامانه‌ی الکترونیکی جامع برای مدیریت تعاونی‌های مسکن و روش انجام کار مطرح شد. در فصل‌های بعدی این گزارش به تفصیل به بررسی مفاهیم پایه و ابزارهایی که در طراحی و پیاده‌سازی این پروژه استفاده شده است، بررسی نیازمندی‌های سیستم شامل نیازمندی‌های کارکردی و غیرکارکردی سامانه، بررسی روند پیاده‌سازی پروژه، نتیجه‌گیری کلی پروژه و پیشنهادات و کارهایی که در آینده می‌توان در این حوزه انجام داد پرداخته شده است.

---

[۱] Admin panel

[۲] Customer panel

[۳] Server

[۴] Micro-Service

[۵] Endpoint

[۶] Front-end

فصل دوم

معرفی مفاهیم پایه و ابزارهای مورد استفاده در پروژه

۲-۱- مفاهیم پایه

در این بخش به بیان مفاهیم پایه‌ای که در ادامه‌ی این نگارش به آن‌ها رجوع می‌شود می‌پردازیم و کاربرد هر یک را توضیح خواهیم داد.

۲-۱-۱- فرانت‌اند

فرانت‌اند به عنوان یکی از دو بخش اصلی در توسعه نرم‌افزار، به تمامی مؤلفه‌های مرتبط با واسط کاربری یک برنامه یا تارنما[۱] اشاره دارد. این قسمت از نرم‌افزار، وظیفه‌ی نمایش اطلاعات به کاربر و دریافت ورودی‌های او را برعهده دارد. فرانت‌اند معمولاً شامل فناوری‌ها و زبان‌های برنامه‌نویسی مختلفی می‌شود که برای ایجاد و اداره واسط کاربری استفاده می‌شوند. این زبان‌ها می‌توانند شامل اچ تی ام ال[۲] سی اس اس[۳] جاوا اسکریپت[۴] و چارچوب‌های مختلف جاوا اسکریپت باشند. فرانت‌اند در تعامل مستقیم با کاربر نقش مهمی ایفا می‌کند و باید طراحی زیبا و کارآمدی داشته باشد تا تجربه کاربری بهینه‌ای ارائه دهد.

۲-۱-۲- بک‌اند[۵]

بک‌اند یکی از دو بخش اصلی در توسعه‌ی نرم‌افزار است و مسئولیت اجرای عملیات پشتیبانی و پردازش داده در یک برنامه را داراست. این قسمت از سامانه وظایف مرتبط با زیرساخت را بر عهده دارد و به تعامل با پایگاه داده، مدیریت سرور، ارتباط با برنامه‌های دیگر و منطق کسب‌وکار برنامه توجه دارد. بک‌اند به وسیله زبان‌ها و چارچوب‌های متنوعی توسعه می‌یابد که به توسعه‌دهندگان این امکان را می‌دهد که لایه‌ی منطقی برنامه را ایجاد و مدیریت کنند. این لایه مسئولیت‌هایی مانند اعتبارسنجی داده، مدیریت

---

[۱] Website

[۲] HTML

[۳] CSS

[۴] Java Script

[۵] Back-end

کاربران، پردازش فرآیندها و ایجاد و ارسال پاسخ‌ها به فرانت‌اند را برعهده دارد. بک‌اند باید از امنیت و پایداری بالا برخوردار باشد و نقش اساسی در عملکرد یک سیستم نرم‌افزاری ایفا کند.

۲-۱-۳- طراحی دامنه محور[۱]

هنگامی که از معماری نرم‌افزار یک سامانه صحبت می کنیم، می خواهیم بدانیم آن سامانه چیست و چگونه رفتار می کند. معماری نرم‌افزار مجموعه ای از ساختارهای مورد نیاز برای پیشبرد و توسعه‌ی نرم‌افزار است که تمام المان‌های نرم‌افزار و رابطه‌ی آن‌ها با یکدیگر و همچنین ویژگی‌های این المان‌ها و رابطه‌ها را مشخص می‌کند. به طور کلی هدف این رویکرد این است که بگوید ساختار و زبان کد یک نرم‌افزار، باید با حوزه‌ی کسب و کار آن مطابقت داشته باشد. از مزایای این معماری می‌توان به تسهیل ارتباطات، انعطاف پذیری و قابلیت نگهداری اشاره کرد.

۲-۱-۴- معماری مبتنی بر سرویس[۲]

این معماری به اپلیکیشن‌ها امکان ارتباط با یکدیگر روی یک رایانه منفرد و یا در زمان توزیع اپلیکیشن‌ها، روی چندین رایانه در یک شبکه را ارائه می‌کند. هر میکروسرویس ارتباط اندکی با سرویس‌های دیگر دارد. این سرویس‌ها خودکفا هستند و یک کارکرد منفرد (یا گروهی از کارکردهای مشترک) را ارائه می‌کنند. معماری میکروسرویس‌ها به طور طبیعی در سازمان‌های بزرگ و پیچیده استفاده می‌شود که در آن‌ها چند تیم توسعه می‌توانند مستقل از هم برای ارائه یک کارکرد تجاری کار کنند و یا اپلیکیشن‌ها ملزم به ارائه خدمات به یک حوزه تجاری باشند.

---

[۱] Domain Driven Design (DDD)

[۲] Services Oriented Architecture (SOA)

۲-۲- معرفی ابزارها و فناوری‌های استفاده شده در پروژه

۲-۲-۱- انگولار[۱]

انگولار یکی از چارچوب‌های محبوب جاوا اسکریپت در فرانت‌اند است. این چارچوب توسط تیم گوگل توسعه یافته و منتشر شده‌است. انگولار به عنوان یک ابزار قدرتمند برای ایجاد وب‌اپلیکیشن‌های تعاملی و پویا مورد استفاده قرار می‌گیرد و ویژگی‌های زیر را داراست:

۱. **مؤلفه‌ها**[۲]: انگولار از مفهوم مؤلفه‌ها برای تقسیم و تجزیه و تحلیل واسط کاربری به قطعات کوچک‌تر استفاده می‌کند. این به توسعه‌دهندگان اجازه می‌دهد که کدهای خود را ماژولار و قابل مدیریت کنند.

۲. **دایرکتیوها**[۳]: انگولار دارای دایرکتیوهایی است که به توسعه‌دهندگان این امکان را می‌دهد که به سادگی ویژگی‌هایی مانند تغییر وضعیت دام[۴]را کنترل کنند.

۳. **واحد تست**: انگولار ابزارهای قدرتمندی برای تست واحد و تست ادغامی[۵]را فراهم می‌کند تا کیفیت برنامه‌های توسعه یافته با این چارچوب را افزایش دهد.

۴. **پشتیبانی از اس پی ای**[۶]:انگولار مناسب برای ساخت برنامه‌های وب تک‌صفحه‌ای است که به بارگذاری مجدد صفحه نیاز ندارند و صفحات به صورت پویا با یک سیستم ناوبری قابل تغییر هستند[۱].

---

۱ Angular

۲ Component

۳ Directive

۴ Document Object Model (DOM)

۵ Integration Test

۶ Single Page Application (SPA)

۲-۲-۲- دات‌نت[۱]

دات‌نت یک چارچوب توسعه نرم‌افزاری است که توسط شرکت مایکروسافت[۲]ایجاد شده‌است و برای توسعه بک‌اند نرم‌افزارها بسیار مورد استفاده قرار می‌گیرد. با دات‌نت می‌توان وب‌اپلیکیشن‌ها، سرویس‌های وب، سرورهای میانی، و نرم‌افزارهای تعاملی و پیچیده را ایجاد کرد. این چارچوب از ابزارها و کتابخانه‌های بسیاری برای پایگاه داده، مدیریت سرور، امنیت، نحوه انتقال داده و بسیاری از نیازهای بک‌اند نرم‌افزارها پشتیبانی می‌کند. دات‌نت به توسعه‌دهندگان امکان ایجاد بک‌اندهای مقیاس‌پذیر و پایدار را می‌دهد و با استفاده از آن، می‌توان وب‌سرویس‌ها و اپلیکیشن‌های وب پویا را ایجاد کرد[۲].

۲-۲-۳- اس کیو ال سرور[۳]

اِس کیو اِل یک زبان استاندارد برای دسترسی و کار با پایگاه‌داده است که آن‌را «زبان ساختاریافته جستار» یا «زبان پرس‌وجوی ساختار یافته» نیز معنی می‌کنند . اس کیو ال سرور یک بانک اطلاعاتی از نوع دیتابیس‌های رابطه‌ای است. وظیفه اصلی آن ذخیره و بازیابی اطلاعات براساس درخواست نرم‌افزارهای دیگر می‌باشد. مهم ترین کاربرد اس کیو ال سرور ساخت بانک اطلاعاتی با حجم بالا است که تعداد زیادی کاربر به طور همزمان می توانند به آن دسترسی داشته باشند.

---

۱ .Net

۲ Microsoft

۳ SQL Server

فصل سوم

جمع آوری نیازمندی‌ها و طراحی سامانه

۳-۱- مقدمه

در این فصل سامانه‌ای طراحی خواهد شد که ویژگی‌های اساسی و لازم برای مدیریت یک سامانه‌ی تعاونی‌های مسکن مناسب را داشته باشد. این ویژگی‌ها عبارت است از:

- صحت داده
- اعتبار سنجی اعضاء
- شفافیت فرایند پیشرفت پروژه‌ها
- امنیت
- طراحی واسط کاربری زیبا
- کارایی

برای بدست آوردن این ویژگی‌ها در سامانه، نیازمندی‌هایی وجود دارد که در قسمت بعدی به شرح آن خواهیم پرداخت.

۳-۲- نیازمندی‌ها

نیازمندی‌ها توصیف‌ها و مشخصاتی هستند که تعیین می‌کنند که سیستم چه ویژگی‌ها، عملکردها و مشخصاتی باید داشته باشد تا به نیازهای مشتریان و کاربران پاسخ دهد و اهداف پروژه را به درستی برآورده سازد. به طور کلی نیازمندی‌های سامانه به دو دسته کلی نیازمندی‌های کارکردی و غیرکارکردی تقسیم می‌شود. سیاست‌های

۳-۲-۱- نیازمندی‌های کارکردی

نیازمندی‌های کارکردی در یک محصول یا سرویس به ویژگی‌های اساسی و عملکردی که به طور مستقیم برای کاربران اهمیت دارد، اشاره دارد. این شامل امکانات اصلی و عملکرد محصول است. نیازمندی‌های کارکردی این سیستم در جدول ۱ به طور کامل شرح داده شده‌است:

جدول ۱ - نیازمندی‌های کارکردی

ردیف	ویژگی	کد نیازمندی	عنوان	شرح
۱	ساخت پروژه	FR01	ایجاد پروژه	کاربر باید بتواند یک پروژه دلخواه را ایجاد کند.
		FR02	افزودن عکس پروژه	کاربر باید بتواند برای پروژه‌ی ایجاد شده عکس آپلود کند.
		FR03	ایجاد واحدها	کاربر باید بتواند واحدهای پروژه را تعریف و اضافه کند.
		FR04	ایجاد فازها	کاربر باید بتواند فازهای پروژه را تعریف و اضافه کند.
		FR05	افزودن عکس فاز	کاربر باید بتواند برای هر فاز عکس‌های مربوطه را آپلود کند.
۲	گزارش پروژه	FR06	مشاهده لیست پروژه‌ها	کاربر باید بتواند لیست تمامی پروژه‌ها را ببیند.
		FR07	مشاهده جزئیات یک پروژه	کاربر باید بتواند صفحه جزئیات یک پروژه را ببیند.
		FR08	مشاهده قراردادها	کاربر باید بتواند قراردادهای یک پروژه را ببیند.
۳	ورود به سامانه	FR09	وارد کردن شماره تماس	کاربر باید بتواند نام کاربری خود را وارد کند.
۴	احراز هویت	FR10	OTP[۱]	کاربر باید بتواند با شماره موبایل خود اهراز هویت شود.

[۱] One-Time Password (OTP)

۵	ایجاد قرارداد	FR11	ایجاد یک قرارداد	کاربر باید بتواند برای واحد موجود مورد نظر قرارداد ثبت کند.
۶	پرداخت	FR12	پرداخت اقساط	کاربر بتواند اقساط قراردادهای خود را پرداخت کند .
۷		FR13	مشاهده سوابق پرداخت	کاربر باید بتواند سوابق پرداخت خود را مشاهده کند.
۸	ویرایش اطلاعات	FR14	ویرایش پروفایل	کاربر باید بتواند برخی اطلاعات خود را ویرایش کند.

۳-۲-۲- نیازمندی‌های غیرکارکردی

نیازمندی‌های غیرکارکردی شامل جنبه‌های غیرعملکردی مانند امنیت، قابلیت اطمینان، قابلیت همگرایی، پشتیبانی و مدیریت منابع (مانند نیاز به پهنای باند، فضای ذخیره‌سازی و سرور)، پشتیبانی از چند زبانی، و قابلیت انطباق با استانداردها و مقررات می‌باشد.

نیازمندی‌های غیرکارکردی این سامانه در جدول ۲ به طور کامل شرح داده شده‌است:

جدول ۲ - نیازمندی‌های غیرکارکردی

ردیف	ویژگی	کد نیازمندی	عنوان	شرح
۱	امنیت	NFR01	محرمانگی داده	حفاظت از داده‌های حساس و اطلاعات کاربران به درستی انجام شود.
		NFR02	اصالت داده	اصالت داده‌ها حفظ شود و داده‌ها تغییر نکنند.
		NFR03	مدیریت هویت و دسترسی	در سامانه باید تعیین شود که کدام کاربران به چه مناطق و قابلیت‌های وبسایت دسترسی دارند.

۲	کارایی	NFR04	سرعت بارگذاری	سرعت بارگذاری صفحات و محتواها بالا باشد.
		NFR05	بهره‌وری منابع سرور	بهره‌وری از منابع سرور، بهینه‌سازی پایگاه‌داده و مدیریت درخواست‌های سرور نقش مهمی در کاهش هزینه‌ها و افزایش کارایی دارند.
۳	تجربه‌ی کاربری	NFR06	طراحی رابط کاربری مناسب	وبسایت باید دارای یک واسط کاربری مناسب و جذاب باشد که با تجربه و انتظارات کاربران هماهنگ باشد.
		NFR07	واکنش‌گرایی	سامانه باید برای تمام اندازه‌ها و دستگاه‌ها به خوبی نمایش داده شود.
		NFR08	ناوبری ساده و منطقی	سیستم ناوبری و منوها باید ساده و منطقی باشند تا کاربران به راحتی به مطالب و صفحات مختلف دسترسی داشته باشند.

۳-۳- انتخاب فناوری

انتخاب فناوری از مهم‌ترین مراحل انجام پروژه است که از سه مرحله‌ی زیر تشکیل شده‌است:

۱- انتخاب ملاک‌ها

۲- بررسی گزینه‌های موجود

۳- مقایسه‌ی گزینه‌ها بر اساس ملاک‌ها

۳-۳-۱- ملاک‌ها

ملاک‌های انتخاب فناوری در پروژه با توجه به نیازمندی‌های مطرح شده در بخش قبل شامل توسعه‌پذیری، آشنا بودن تیم فنی با فناوری، پاسخگوی نیازمندی‌ها و از رده خارج نبودن می‌شود. این ملاک‌ها را به شرح زیر می‌باشد:

- **توسعه‌پذیری**

فناوری‌های انتخابی باید قابلیت توسعه و افزایش امکانات را داشته باشند. این به معناست که اگر پروژه در آینده نیاز به افزودن ویژگی‌های جدید یا مقیاس‌پذیری داشته باشد، فناوری انتخابی باید این امکان را فراهم کند.

- **آشنا بودن تیم فنی با فناوری**

توانایی تیم فنی در استفاده از فناوری‌های انتخاب شده بسیار حیاتی است. اگر تیم فنی تجربه کافی در توسعه و پیاده‌سازی فناوری‌های انتخابی نداشته باشد، سرعت توسعه و کیفیت کار در پروژه کاهش می‌یابد.

- **پاسخگوی نیازمندی‌ها**

فناوری باید قابلیت ایجاد و پیاده‌سازی ویژگی‌ها و امکانات مورد نیاز پروژه را فراهم کند.

- **از رده خارج نبودن**

فناوری انتخابی باید به مرور زمان به‌روزرسانی‌ها و پشتیبانی‌های لازم را از تولیدکنندگان خود داشته باشد تا از از کم شدن امنیت و کارایی جلوگیری کند.

۳-۳-۲- مقایسه و انتخاب فناوری

با توجه به نیازمندی‌های پروژه و ملاک‌های انتخاب شده، از فناوری‌های زیر برای قسمت‌های مختلف استفاده شده است:

۳-۳-۳- فرانت‌اند

در ابتدا به عنوان گزینه‌ها چارچوب‌های ری‌اکت و انگولار مورد بررسی قرار گرفته شد. پس از بررسی و تحلیل نیازمندی‌های پروژه تصمیم به استفاده از انگولار گرفته شد. این انتخاب مبتنی بر داشتن تجربه کاری و توانایی توسعه در این چارچوب می‌باشد. انگولار از مزایایی مانند پشتیبانی از تایپ‌اسکریپت و امکانات قدرتمند برای مدیریت واسط‌های کاربری بهره‌مند است.

۳-۳-۴- بک‌اند

برای بخش بک‌اند پروژه، تصمیم به استفاده از دات‌نت گرفته شد. این انتخاب نیز مبتنی بر داشتن تجربه در این چارچوب و زبان برنامه‌نویسی است. دات‌نت به عنوان یک چارچوب محبوب و قدرتمند برای توسعه وب‌سرویس‌ها و برنامه‌های تحت وب شناخته می‌شود و به خوبی توانایی امنیتی و مقیاس‌پذیری را ارائه می‌دهد.

۳-۴- طراحی

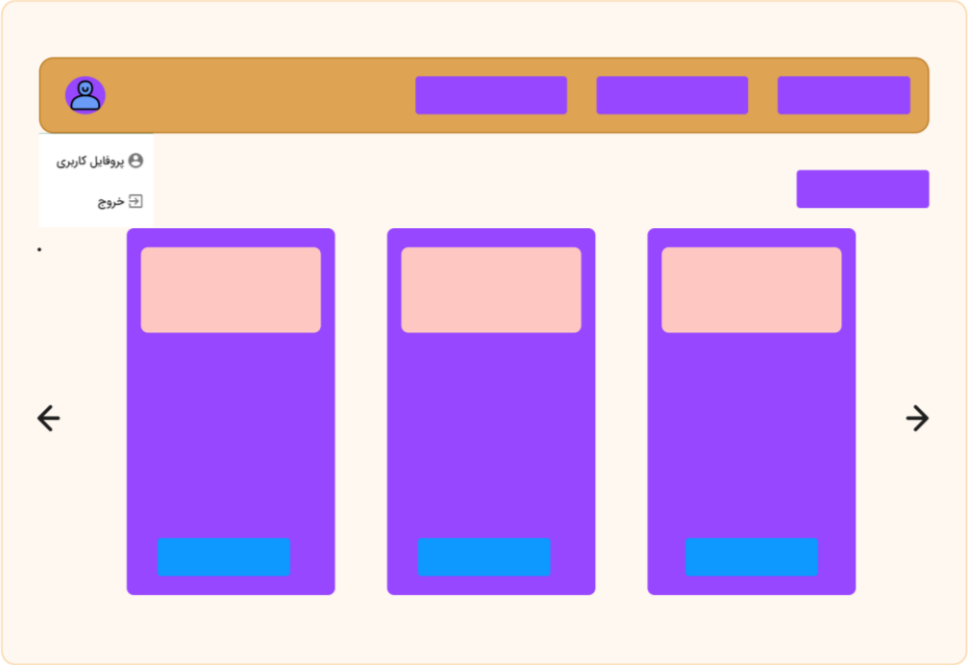
در این بخش به توضیح مراحل طی شده برای طراحی پرداخته شده‌است. این مراحل با انتخاب فناوری مناسب شروع شده و سپس به طور دقیق‌تر گام‌های طی شده برای طراحی پروژه فعلی بررسی می‌شود.

۳-۴-۱- طراحی واسط کاربری

طراحی واسط کاربری در یک پروژه بسیار اهمیت دارد، زیرا نقش بسیار بزرگی در تجربه کاربری[۱] ایفا می‌کند. در واقع، واسط کاربری می‌تواند تأثیر مستقیمی بر موفقیت یا شکست یک نرم‌افزار یا تارنما داشته باشد. از مزایای یک واسط کاربری مناسب می‌توان به تجربه‌ی کاربری بهتر، افزایش رضایت کاربران، کاهش خطاها و اشتباهات و افزایش کارایی اشاره کرد.

[۱] User Experience

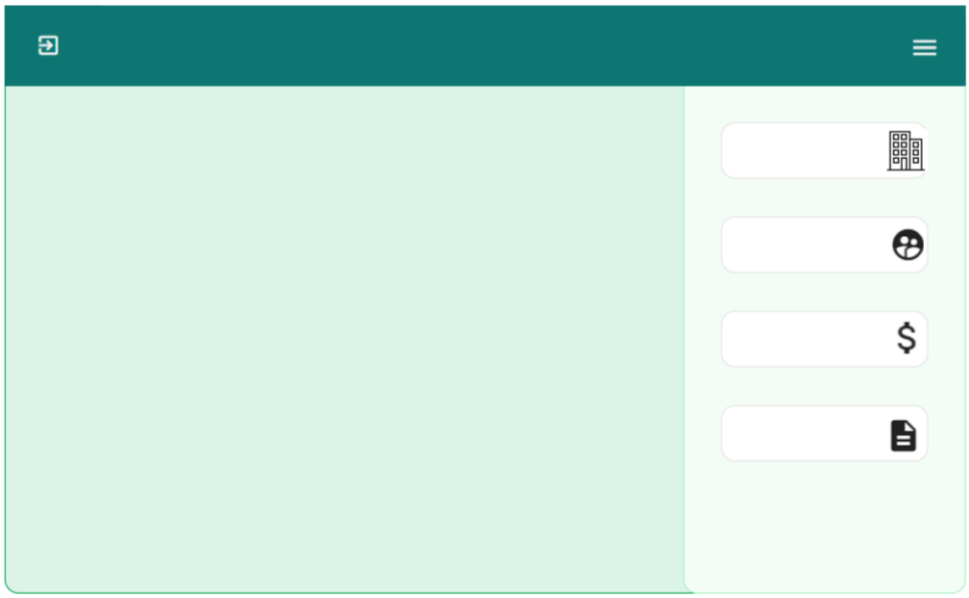
طرح اولیه شامل یک طراحی بدون جزییات است که تنها در آن ساختار کلی بخش‌های مختلف نمایش داده می‌شود. این طراحی می‌تواند بصورت دستی و روی کاغذ کشیده شده و یا از طریق نرم‌افزارهایی مانند فیگما[1]ترسیم شود. بطور کلی به این طراحی خام، وایرفریم[2]هم گفته می‌شود. از آنجایی که وایرفریم شامل جزییات نمی‌شود، هنگام پیاده‌سازی اصلی می‌تواند بنا به نیاز دچار تغییر شده تا بهترین محصول ممکن شکل بگیرد. در پروژه فعلی از خود فیگما برای طراحی وایرفریم استفاده کردیم که دو نمونه آن در شکل ۱ و شکل ۲ قابل مشاهده است. این نمونه‌ها مربوط به صفحه لندینگ پنل کاربری و پنل ادمین است و همانطور که در ادامه نشان خواهیم داد، طراحی اصلی از نظر ظاهری با وایرفریم متفاوت است اما چیزی که وایرفریم و طراحی اصلی در آن مشترک هستند، کلیت ورودی‌های مورد نیاز است.



شکل ۱- صفحه فرود پنل کاربری

[1] Figma

[2] Wireframe



شکل ۲ - صفحه فرود پنل مدیریت

پس از تایید و نهایی شدن وایرفریم، نوبت به طراحی اصلی می‌رسد. خروجی این بخش همان طرحی است که به دست برنامه‌نویس برای اجرا می‌رسد و کاربر نهایی آن را می‌بیند. در این قسمت تمامی جزییات مهم بوده و پیاده می‌شوند.

جزییات طراحی شامل تعیین رنگ هر بخش، حالت‌های مختلف هر مؤلفه مانند هاور[1] و فوکوس[2]، فاصله‌ی بین مؤلفه‌ها به پیکسل، اتفاقی که بعد از کلیک کردن هر دکمه رخ می‌دهد و مواردی از این دست با حداکثر دقت پیاده می‌شوند.

با توجه به وقت‌گیر بودن پیاده‌سازی این حجم از جزییات و تکراری بودن بسیاری از بخش‌ها، بسیار رایج است که طراح چند صفحه‌ی مهم را طراحی می‌کند و اگر باقی صفحات از الگوی فعلی پیروی کنند، آن‌ها را با این جزییات پیاده نمی‌کند و به وایرفریم بسنده می‌کند.

[1] Hover

[2] Focus

۳-۴-۲- طراحی فرانت‌اند

در طراحی بخش فرانت‌اند سامانه، بخش‌های مختلف عملکردی را بـه قسـمت‌های کوچـک‌تر بـا وظـایف مشخص به نام مؤلفه تقسیم می‌کنیم. این رویکرد تجزیه و تحلیل مؤلفه‌ای[۱]نام دارد. این رویکرد مزایـای زیادی دارد که در ادامه شرح داده شده‌است:

۱. **مدیریت کد**: این مؤلفه‌ها می‌توانند مستقل از یکدیگر توسعه داده شـده و تسـت شـوند. ایـن باعـث می‌شود که مدیریت کد آسان‌تر شود و بهتر قابل توسعه باشد.

۲. **تعامل و هماهنگی**: از آنجا که هر مؤلفه مستقل عمل می‌کند، توسعه‌دهندگان می‌توانند به تعامـل و هماهنگی بهتری بین مؤلفه‌ها دست یابند. این باعث کاهش اشکالات و مشکلات احتمالی در تعامل کدها می‌شود.

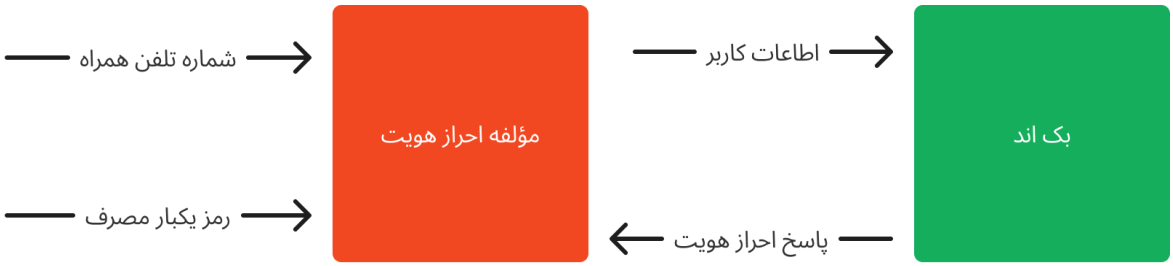
۳. **قابلیت استفاده مجدد**: مؤلفه‌های کوچک‌تر به راحتی می‌توانند در پروژه‌های دیگر اسـتفاده شـوند. این قابلیت استفاده مجدد کد، زمان توسعه را کاهش می‌دهد و به افزایش بهره‌وری کمک می‌کند.

۴. **تست و اشکال‌زدایی آسان‌تر**: با تجزیه پروژه به مؤلفه‌های کوچک‌تر، تست و اشکال‌زدایی آسـان‌تر می‌شود. می‌توان هر مؤلفه را به صورت مستقل تست کرد و به راحتی مشکلات را پیدا و رفـع کـرد. ایـن مؤلفه‌ها در کنار یکدیگر صفحات قابل مشاهده نرم‌افزار را بـرای کـاربر ایجـاد می‌کننـد. نرم‌افـزار بخـش فرانت‌اند به طور کلی از مؤلفه‌های زیر تشکیل می‌شود:

- **مؤلفه احراز هویت کاربر**

وظیفه این مؤلفه دریافت شماره تلفن همراه از کاربر و ارسال به بک‌اند است. سپس رمز عبور یک‌بار مصرف برای کاربر ارسال می‌شود و کاربر رمز را وارد می‌کند. در صورت احراز هویت، اجازه ورود داده می‌شود و در غیر این صورت از ورود کاربر جلوگیری می‌شود. عملکرد کلی این مؤلفه در شکل ۳ قابل مشاهده است.

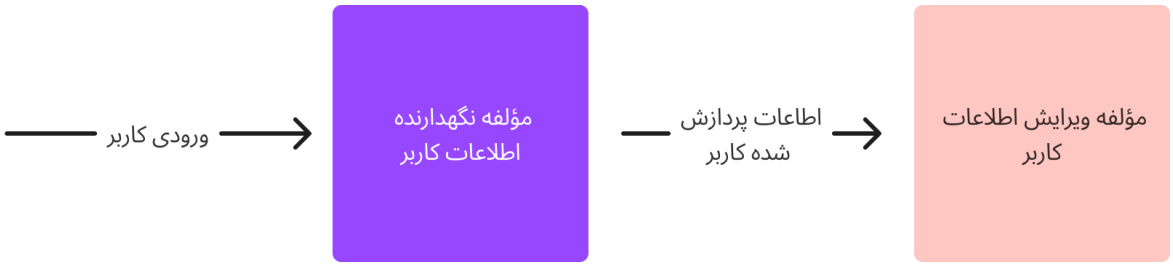
[۱] Component-Based Architecture



شکل ۳ - عملکرد کلی مؤلفه احراز هویت کاربر

- **مؤلفه نگهدارنده اطلاعات کاربر**

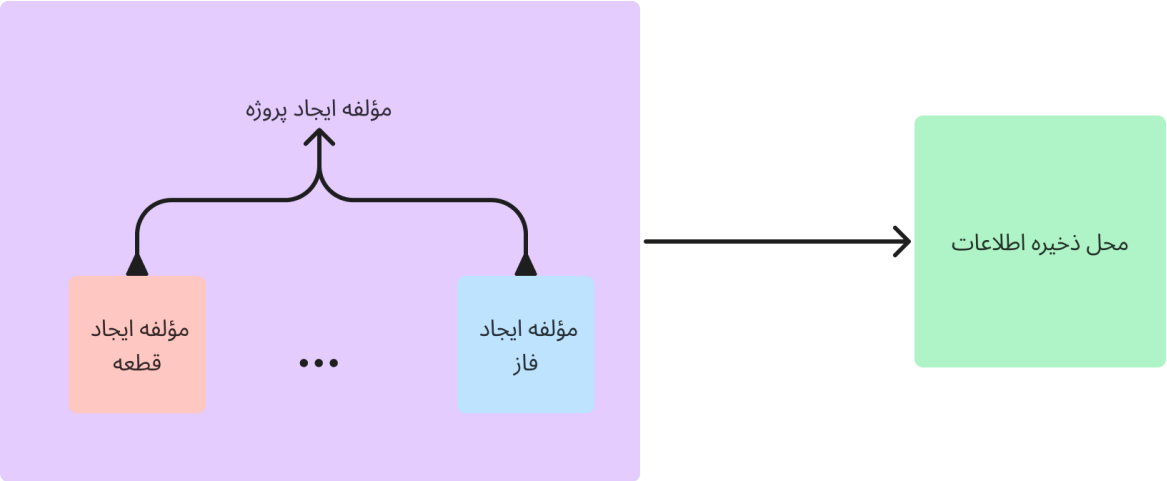
این مؤلفه وظیفه دارد یک قالب از پیش تعریف شده برای دریافت اطلاعات کاربر را به او نمایش داده و اطلاعات ورودی کاربر را جهت ویرایش دریافت کنند. این اطلاعات شامل نام، نام خانوادگی، کدملی، تاریخ تولد، جنسیت و دیگر اطلاعات ضروری کاربر است. با هر ورودی کاربر این مؤلفه اطلاعات را به مؤلفه میزبان خود ارسال می‌کند. عملکرد کلی این مؤلفه در شکل ۴ قابل مشاهده است.



شکل ۴ - عملکرد کلی مؤلفه نگه دارنده اطلاعات کاربر

- **مؤلفه ایجاد پروژه:**

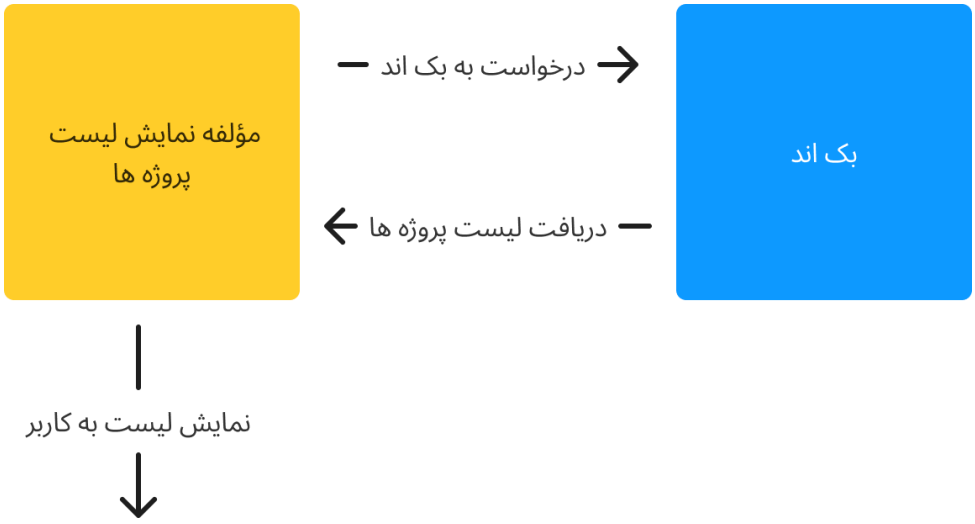
وظیفه نگهداری و نمایش لیست مؤلفه‌های پروژه و ایجاد یک پل ارتباطی بین آن‌ها و بخش ذخیره اطلاعات بر عهده‌ی این مؤلفه است. این مؤلفه مسئول ایجاد پروژه جدید یا ویرایش یک پروژه از پیش ساخته شده‌است. عملکرد کلی این مؤلفه در شکل ۵ قابل مشاهده است.



شکل ۵ - عملکرد کلی مؤلفه‌های ایجاد پروژه

- **مؤلفه‌ی نمایش لیست همه‌ی پروژه‌ها**

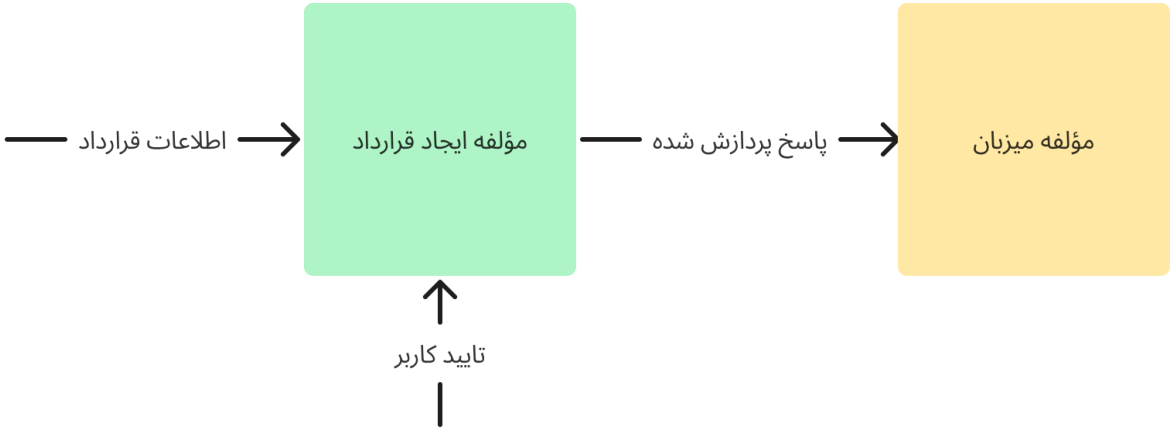
این مؤلفه وظیفه‌ی دریافت اطلاعات از بک‌اند و نمایش لیست پروژه‌ها به کاربر را دارد. عملکرد کلی این مؤلفه در شکل ۶ قابل مشاهده است.



شکل ۶ - عملکرد کلی نمایش لیست همه‌ی پروژه‌ها

- **مؤلفه‌ی ایجاد قرارداد**

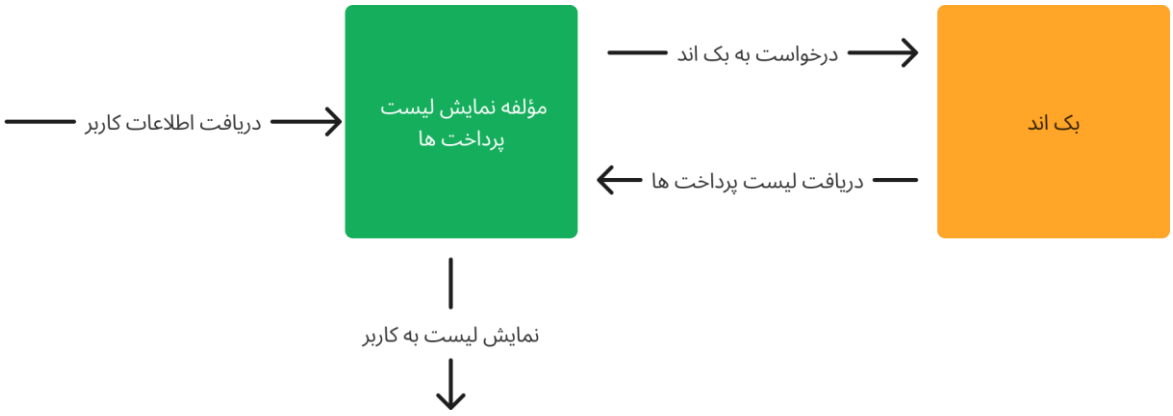
این مؤلفه در پنل کاربری قرار دارد. وظیفه نمایش اطلاعات قرارداد به کاربر و ایجاد قرارداد بر عهده‌ی این مؤلفه است. عملکرد کلی این مؤلفه‌ها در شکل ۷ قابل مشاهده است.



شکل ۷ - عملکرد کلی مؤلفه‌ی ایجاد قرارداد

- **مؤلفه‌ی نمایش همه‌ی پرداخت‌های کاربر**

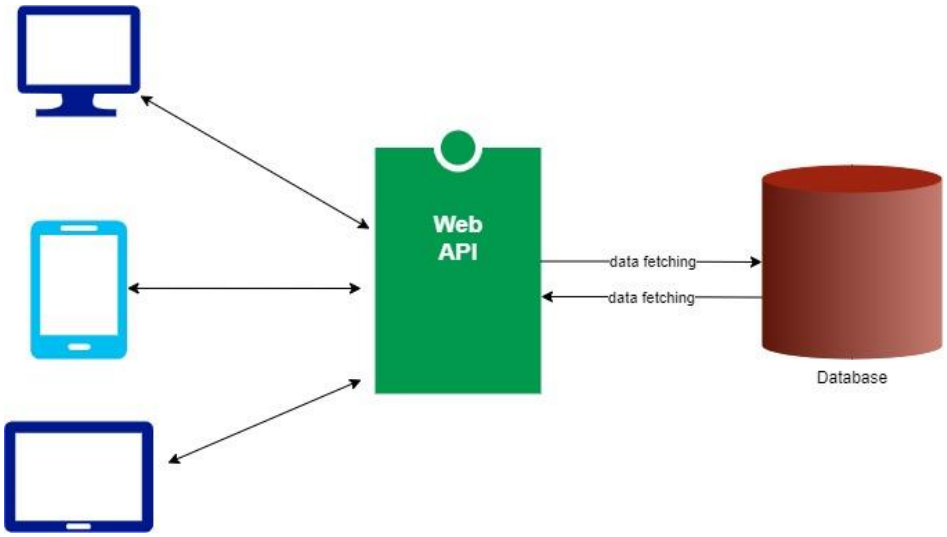
این مؤلفه نیز در پنل کاربری قرار دارد. وظیفه‌ی آن نمایش ترتیبی پرداخت‌های کاربر به او است. عملکرد کلی این مؤلفه در شکل ۸ قابل مشاهده است.



شکل ۸ - عملکرد کلی نمایش لیست پرداخت‌های یک کاربر

۳-۴-۳- طراحی بک‌اند

در راستای طراحی بک‌اند باید به طراحی جدول‌های پایگاه داده بپردازیم. در بخش بک‌اند چند نقطه‌ی انتهایی در نظر گرفته شده‌است که واسط بین رابط کاربری و بک‌اند می‌باشد و سپس اطلاعات دریافت شده به پایگاه داده ارسال می‌شود. اطلاعات ارسال شده در جدول‌های مربوطه ذخیره می‌شود.



شکل ۹ - نحوه‌ی کارکرد بک‌اند**[۳]**

نحوه‌ی ذخیره‌ی داده‌ها

برای پیاده‌سازی این فرم‌ساز موجودیت‌های کاربر، پروژه، قرارداد، قطعه، فاز، مورد قرارداد و مورد پرداخت شده‌ی قرارداد تعریف شده‌است. ساختار کلی اطلاعات به این صورت است که هر پروژه از چندین واحد با قیمت‌ها و ویژگی‌های متفاوت تشکیل شده‌است. همچنین هر پروژه در روند پیشرفت خود چندین فاز دارد. هر کاربر در چندین پروژه مشارکت دارد، که این مشارکت‌ها بوسیله‌ی قراردادهای او تعیین می‌شود. در تمامی جدول‌ها ۵ مورد مشترک است :

۱- آی دی جدول که همان کلید اصلی[۱]جدول است

۲- تاریخ ساخته شدن که تاریخ زمانی است که این سطر در جدول ثبت شده‌است

۳- تاریخ آخرین به‌روزرسانی که تاریخ اخرین زمانی است که اطلاعات این سطر از جدول به‌روزرسانی شده‌است.

۴- وضعیت حذف شدن سطر که نشان می‌دهد آیا این سطر از داده‌ها حذف شده‌است یا خیر.

۵- وضعیت قابل مشاهده بودن سطر که نشان می‌دهد آیا این سطر از داده‌ها قابل مشاهده است یا خیر.

[۱] Primary key

محتوای جدول‌ها به شرح زیر است.

- **جدول کاربر**

این جدول برای نگهداری اطلاعات کلی یک کاربر استفاده می‌شود. به ازای هر کاربر جدید که ثبت نام می‌کند، یک سطر جدید ثبت می‌شود که شامل موارد زیر است:

Name	Type	Size	Scale	Not null	Key
Id	nvarchar	450		☑	1
FirstName	nvarchar(max)			☐	
LastName	nvarchar(max)			☐	
PhoneNumber	nvarchar(max)			☑	
NationalId	nvarchar(max)			☐	
BirthDate	datetime2	7		☐	
Gender	int			☐	
JobTitle	nvarchar(max)			☐	
City	nvarchar(max)			☐	
Address	nvarchar(max)			☐	
CreatedAt	datetime2	7		☑	
ModifiedAt	datetime2	7		☐	
IsDeleted	bit			☑	
IsVisibled	bit			☑	

شکل ۱۰ - طراحی جدول کاربر

۱- آی دی کاربر

۲- نام کاربر

۳- نام خانوادگی کاربر

۴- شماره تلفن همراه کاربر

۵- کد ملی کاربر

۶- تاریخ تولد کاربر

۷- جنسیت کاربر

۸- عنوان شغلی کاربر

۹- شهر محل سکونت کاربر

۱۰- آدرس محل سکونت کاربر

- **جدول پروژه**

این جدول برای نگهداری اطلاعات یک پروژه می‌شود و به ازای هر پروژه که توسط مدیر ساخته می‌شود یک سطر جدید ثبت می‌شود که شامل موارد زیر است:

Name	Type	Size	Scale	Not null	Key
Id	nvarchar	450		☑	🔑1
Address	nvarchar(max)			☐	
Type	int			☐	
ImageIds	nvarchar(max)			☐	
EngineerName	nvarchar(max)			☐	
Description	nvarchar(max)			☐	
CreatedAt	datetime2	7		☑	
ModifiedAt	datetime2	7		☐	
IsDeleted	bit			☑	
IsVisibled	bit			☑	
EndDate	datetime2	7		☐	
EstimatedEndDate	datetime2	7		☐	
EstimatedStartDate	datetime2	7		☐	
Name	nvarchar(max)			☐	
StartDate	datetime2	7		☐	

شکل ۱۱ - طراحی جدول پروژه

۱- آی دی پروژه

۲- آدرس محل ساخت پروژه

۳- نوع پروژه (مسکونی- زمین)

۴- لیست آی دی عکس‌های پروژه

۵- نام و نام خانوادگی مهندس پروژه

۶- توضیحات پروژه

۷- نام پروژه

۸- تاریخ تقریبی شروع به ساخت پروژه

۹- تاریخ تقریبی پایان ساخت پروژه

۱۰- تاریخ واقعی شروع ساخت پروژه

۱۱- تاریخ واقعی پایان ساخت پروژه

- **جدول قطعه**

این جدول برای نگهداری اطلاعات قطعات پروژه‌ها استفاده می‌شود و به ازای هر قطعه مربوط به هر پروژه یک سطر جدید ثبت می‌شود که شامل موارد زیر است:

Name	Type	Size	Scale	Not null	Key
Id	nvarchar	450		✓	1
ProjectId	nvarchar	450		✓	
Meterage	bigint				
Value	bigint				
Description	nvarchar(max)				
CreatedAt	datetime2	7		✓	
ModifiedAt	datetime2	7			
IsDeleted	bit			✓	
IsVisibled	bit			✓	
InstalmentAmount	bigint				
PrePaymentAmount	bigint				
ContractId	nvarchar(max)				
Name	nvarchar(max)			✓	

شکل ۱۲ - طراحی جدول قطعه

۱- آی دی قطعه

۲- آی دی پروژه

۳- متراژ قطعه

۴- قیمت کلی قطعه

۵- توضیحات قطعه

۶- مبلغ پیش پرداخت

۷- مبلغ قسط ماهیانه

۸- آی دی قرارداد

۹- نام قطعه

- **جدول فاز**

این جدول برای نگهداری اطلاعات فازهای مختلف هر پروژه استفاده می‌شود و به ازای هر فاز مربوط بـه هر پروژه یک سطر جدید ثبت می‌شود که شامل موارد زیر است:

Name	Type	Size	Scale	Not null	Key
Id	nvarchar	450		✓	1
ProjectId	nvarchar	450		✓	
Name	nvarchar(max)			☐	
Status	int			☐	
Progress	real			☐	
StartDate	datetime2	7		☐	
EstimatedStartDate	datetime2	7		☐	
ImageIds	nvarchar(max)			☐	
CreatedAt	datetime2	7		✓	
ModifiedAt	datetime2	7		☐	
IsDeleted	bit			✓	
IsVisibled	bit			✓	
Description	nvarchar(max)			☐	
EndDate	datetime2	7		☐	
EstimatedEndDate	datetime2	7		☐	
Order	int			☐	

شکل ۱۳ - طراحی جدول فاز

۱- آی دی فاز

۲- آی دی پروژه

۳- نام فاز

۴- وضعیت فاز (شروع نشده- در حال انجام – پایان یافته)

۵- درصد پیشرفت فاز

۶- تاریخ دقیق شروع فاز

۷- تاریخ تقریبی شروع فاز

۸- تاریخ دقیق پایان فاز

۹- تاریخ تقریبی پایان فاز

۱۰- لیست آی دی عکس‌های پروژه

۱۱- ترتیب فاز

۱۲- توضیحات فاز

- **جدول قرارداد**

این جدول برای نگهداری اطلاعات قرارداد و ارتباط بین قطعه پروژه و کاربر استفاده می‌شود و به ازای هر قطعه مربوط به هر پروژه و کاربری که قصد خرید آن قطعه را دارد، یک سطر جدید ثبت می‌شود که شامل موارد زیر است:

Name	Type	Size	Scale	Not null	Key
Id	nvarchar	450		✓	1
CustomerId	nvarchar	450		✓	
PlotId	nvarchar	450		✓	
ProjectId	nvarchar	450		✓	
TotalCommitedAmount	bigint				
TotalPaidAmount	bigint				
PrePaymentAmount	bigint				
InstalmentAmount	bigint				
Description	nvarchar(max)				
StartDate	datetime2	7			
EndDate	datetime2	7			
CreatedAt	datetime2	7		✓	
ModifiedAt	datetime2	7			
IsDeleted	bit			✓	
IsVisibled	bit			✓	

شکل ۱۴ - طراحی جدول قرارداد

۱- آی دی قرارداد

۲- آی دی کاربر

۳- آی دی قطعه

۴- آی دی پروژه

۵- جمع مبلغی که کاربر باید در طی قرارداد پرداخت کند

۶- جمع مبلغی که کاربر تا کنون برای این قرارداد پرداخت کرده‌است

۷-مقدار پیش پرداخت کاربر برای این قرارداد

۸- مقدار قسط ماهیانه‌ی کاربر برای این قرارداد

۹- توضیحات قرارداد

۱۰- تاریخ شروع قرارداد

۱۱- تاریخ پایان قرارداد

- **جدول قسط قرارداد**

این جدول برای اطلاعات اقساط ماهیانه قرارداد استفاده می‌شود و به ازای هر قرارداد، به تعداد ماه‌هایی که کاربر باید برای این قرارداد قسط پرداخت کند، سطر جدید ثبت می‌شود که شامل موارد زیر است:

Name	Type	Size	Scale	Not null	Key
Id	nvarchar	450		✓	🔑1
ContractId	nvarchar	450		✓	
InstalmentAmount	bigint			☐	
PaidAmount	bigint			☐	
DueDate	datetime2	7		☐	
CreatedAt	datetime2	7		✓	
ModifiedAt	datetime2	7		☐	
IsDeleted	bit			✓	
IsVisibled	bit			✓	

شکل ۱۵ - طراحی جدول قسط قرارداد

۱- آی دی قسط

۲- آی دی قرارداد

۳- مقدار پولی که برای این قسط باید پرداخت شود

۴- مقدار پولی که تا الان برای این قسط پرداخت شده‌است

۵- مهلت پرداخت کامل قسط

- **جدول قسط پرداخت شده قرارداد**

این جدول برای پرداخت‌هایی که برای هر قسط انجام شده، استفاده می‌شود و به ازای هر پرداخت بـرای یک قسط، یک سطر جدید ثبت می‌شود که شامل موارد زیر است:

Name	Type	Size	Scale	Not null	Key
Id	nvarchar	450		✓	1
ContractItemId	nvarchar	450		✓	
Amount	bigint			☐	
PaymentId	nvarchar(max)			✓	
CreatedAt	datetime2	7		✓	
ModifiedAt	datetime2	7		☐	
IsDeleted	bit			✓	
IsVisibled	bit			✓	

شکل ۳-۳ طراحی جدول قسط پرداخت شده قرارداد

۱- آی دی پرداخت برای این قسط

۲- آی دی قسط

۳- مبلغ پرداخت شده

۴- آی دی پرداخت

۳-۵- جمع بندی

در این بخش ابتدا انتخاب فناوری مورد بررسی قرار گرفت. ملاک‌های مورد توجه مطرح شد و با توجه به ملاک‌ها فناوری مناسب انتخاب شد. سپس طراحی سامانه در سه بخش واسط کاربری، فرانت‌اند و بک‌اند شرح داده شد. در بخش واسط کاربری طرح‌های فیگما، در بخش فرانت‌اند مؤلفه‌های کاربری و در بخش بک‌اند ساختار جداول طراحی شد.

فصل چهارم

پیاده‌سازی

۴-۱- مقدمه

سامانه‌ی مدیریت تعاونی‌های مسکن پیاده‌سازی شده به این شکل عمل می‌کند که پروژه‌های مسکونی به همراه واحدهایشان توسط کاربران پنل مدیریت ساخته می‌شوند. کاربران پنل کاربری با مشاهده‌ی لیست پروژه‌ها و واحدهای موجود می‌توانند برای واحدهای مورد نظر خود قرارداد ثبت کنند و سپس با توجه به مبلغ پیش پرداخت و مبلغ اقساط ماهیانه‌ی هر قرارداد، اقساط خود را پرداخت کنند. در نهایت با به اتمام رسیدن پروژه و تسویه‌ی حساب کامل، کاربر مالک واحد خود می‌شود. در کل دو پنل داریم: پنل مدیریت و پنل کاربری. در پنل مدیریت، مدیریت ایجاد پروژه‌ها، نگهداری لیست مشارکت‌کننده‌ها و لیست پرداخت‌ها و در پنل کاربری، ثبت قرارداد برای پروژه‌ها، پرداخت اقساط، مشاهده تمامی پروژه‌ها، پروژه‌های کاربر، قراردادهای کاربر، پرداخت‌های کاربر و مشاهده و ویرایش اطلاعات کاربر صورت می‌گیرد.

۴-۲- پیاده‌سازی فرانت‌اند

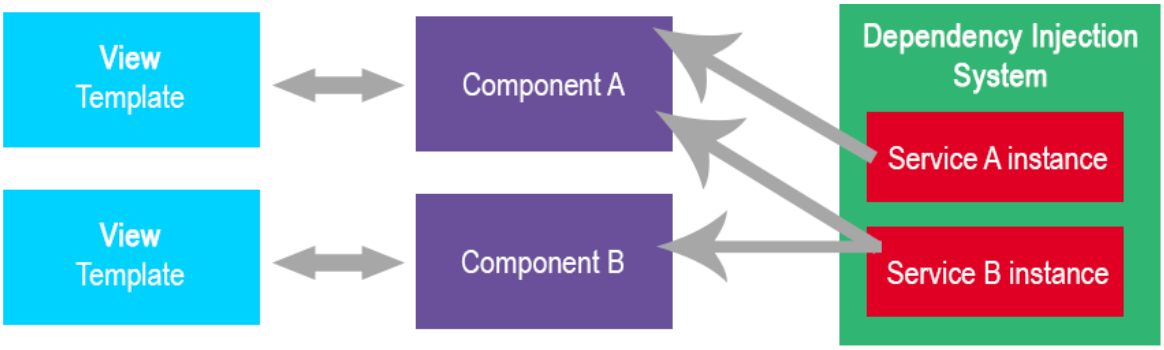
همانطور که در بخش طراحی گفته شد، چارچوب انتخابی برای توسعه فرانت‌اند، چارچوب انگولار است. در پیاده‌سازی از یک الگوی طراحی بسیار کاربردی به نام تزریق وابستگی[۱] که یکی از ویژگی‌های مهم چارچوب انگولار می‌باشد استفاده شده‌است. از آنجا که در اغلب بخش‌های مختلف پیاده‌سازی از این مفهوم استفاده شده‌است در ادامه به صورت مختصر این ویژگی شرح داده می‌شود.

در چارچوب انگولار، تزریق وابستگی یک الگوی مهم است که برای مدیریت وابستگی‌ها بین مؤلفه‌ها و سرویس‌ها استفاده می‌شود. این الگو به توسعه‌دهندگان این امکان را می‌دهد تا به جای اینکه مؤلفه‌ها خودشان این وابستگی‌ها را ایجاد یا مدیریت کنند، وابستگی‌های مورد نیاز مؤلفه‌ها را به صورت تزریق شده از طریق سیستم تزریق وابستگی دریافت کنند.

این رویکرد به بهبود قابلیت تست‌پذیری کد، کاهش اتصال مستقیم بین مؤلفه‌ها، افزایش قابلیت تغییرپذیری و ایجاد کدی تمیزتر کمک می‌کند. در انگولار، تزریق وابستگی به صورت خودکار انجام

[۱] Dependency Injection

می‌شود و تزریق وابستگی‌ها از طریق تزریق‌کننده‌ها[1] انجام می‌شود. نحوه‌ی کار این الگو در شکل ۱۶ نمایش داده شده‌است.



شکل ۱۶ – نحوه‌ی کار الگوی تزریق وابستگی در انگولار**[۴]**

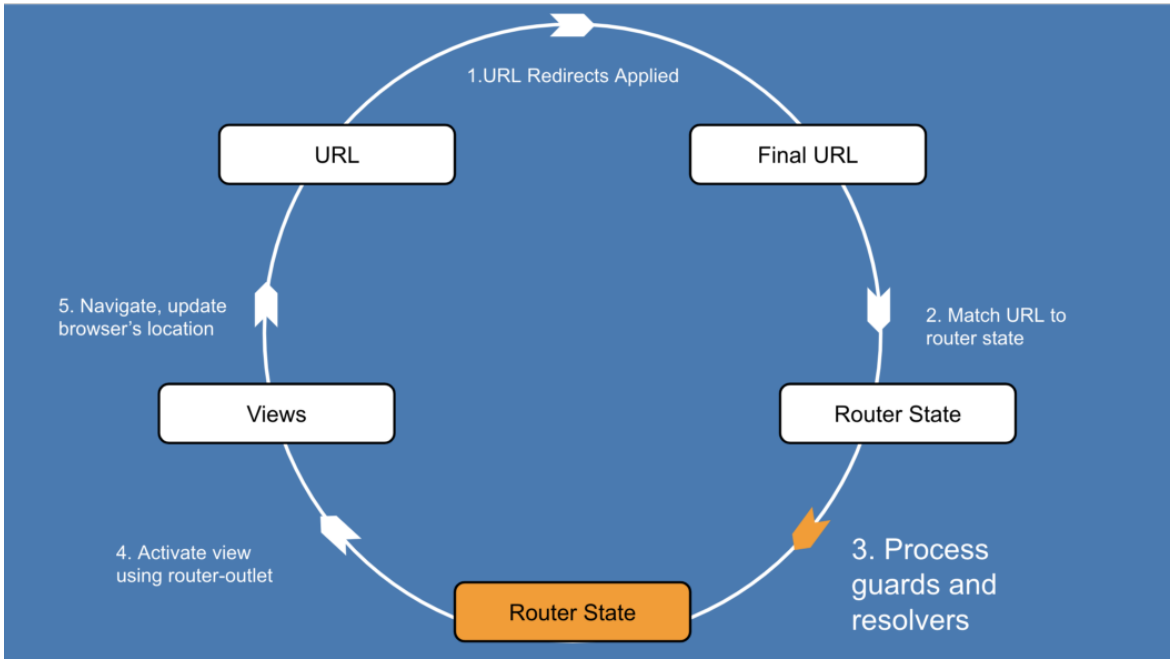
در ادامه بخش‌های مهم توسعه را به طور مجزا عنوان می‌کنیم.

۴-۲-۱- پیاده‌سازی فرانت‌اند

ناوبری در یک برنامه انگولار یکی از جنبه‌های بسیار مهم است که به کاربران این امکان را می‌دهد تا از یک صفحه به صفحه‌های دیگر برنامه منتقل شوند. برنامه‌های انگولار از یک فرایند مسیریابی برای مدیریت ناوبری استفاده می‌کنند که اجازه می‌دهد تا زمانی که کاربر به یک یو آر ال[2] خاص دسترسی پیدا می‌کند یا روی یک لینک کلیک می‌کند، صفحه مربوطه به صورت پویا و بدون نیاز به بارگذاری دوباره صفحه کاملاً تغییر کند. انگولار برای این کار از یک واحد ناوبری استفاده می‌کند که کلاس‌های اصلی آن قابل تزریق به مؤلفه‌های نرم‌افزار است. چرخه عملکرد این واحد در شکل ۱۷ قابل مشاهده است[۵].

[1] Injectors

[2] Uniform Resource Locator (URL)



شکل ۱۷ - چرخه‌ی ناوبری انگولار[۵]

۴-۲-۲- کتابخانه‌ی واسط کاربری

انگولار متریال[۱] یک کتابخانه‌ی طراحی واسط کاربری برای چارچوب انگولار است. این کتابخانه به توسعه‌دهندگان این امکان را می‌دهد تا به راحتی و با استفاده از اجزای آماده‌ای که ارائه می‌دهد، واسط‌های کاربری زیبا و حرفه‌ای بسازند. انگولار متریال از تجهیزات طراحی و پیاده‌سازی رایج استفاده می‌کند تا نرم‌افزارهای وب به راحتی قابل تنظیم و جذاب باشند. در پیاده‌سازی این پروژه نیز از مؤلفه‌های قابل توسعه‌ی این کتابخانه استفاده شده‌است.

۴-۲-۳- کتابخانه‌ی مدیریت وضعیت[۲]

ان جی آر اکس[۳] یک کتابخانه در انگولار است که بر پایه‌ی الگوی ریداکس[۱] ساخته شده‌است. این کتابخانه به توسعه‌دهندگان امکان مدیریت وضعیت برنامه‌های وب انگولار را با استفاده از یک پیش‌فرض

---

[۱] Angular Material

[۲] State Management

[۳] ngrx

یکپارچه و قابل پیش‌بینی ارائه می‌کند. در انگولار، برنامه‌های وب معمولاً دارای وضعیت‌های مختلف و اطلاعاتی هستند که نیاز است به طور مداوم به‌روزرسانی و به اشتراک گذاشته شوند. ان جی آر اکس این وضعیت‌ها را در یک دسته‌بندی یکپارچه و مدیریت می‌کند، که به توسعه‌دهندگان این امکان را می‌دهد که وضعیت برنامه را به صورت پیش‌بینی‌پذیر و قابل مدیریت، مدیریت کنند. در این پروژه نیز فرایندهایی با بهره‌گیری از این کتابخانه صورت گرفته است.

بخش‌های اصلی ان جی آر اکس در انگولار عبارتند از:

۱. مخزن:[۲] اطلاعات و وضعیت برنامه در یک مخزن مرکزی مدیریت می‌شود. این مخزن حاوی وضعیت برنامه است که تمامی اجزا و مولفه‌ها به آن دسترسی دارند.

۲. اعمال:[۳] از اعمال برای تغییر وضعیت استفاده می‌کند. هر عمل یک وضعیت معین را تعیین می‌کند و این اعمال توسط توابع کنترل می‌شوند.

۳. کاهنده‌ها:[۴] کاهنده‌ها در ان جی آر اکس توابعی هستند که بر اساس نوع اعمال و وضعیت فعلی، وضعیت جدید را تولید و اعمال می‌کنند. این توابع به تغییر وضعیت قابل پیش‌بینی کمک می‌کنند.

۴. افعال جانبی:[۵] افعال جانبی برای مدیریت و پردازش افعال جانبی نظیر فراخوانی وب سرویس‌ها، درخواست‌های اچ‌تی‌تی‌پی، یا اعمال دیگری که به طور ناهمگام انجام می‌شوند، مورد استفاده قرار می‌گیرند. این افعال جانبی باعث می‌شوند که تغییرات در وضعیت رخ دهند.

طراحی این کتابخانه همانطور که پیش‌تر بیان شد بر اساس الگوی ریداکس است که در شکل ۱۸ قابل مشاهده است.

---

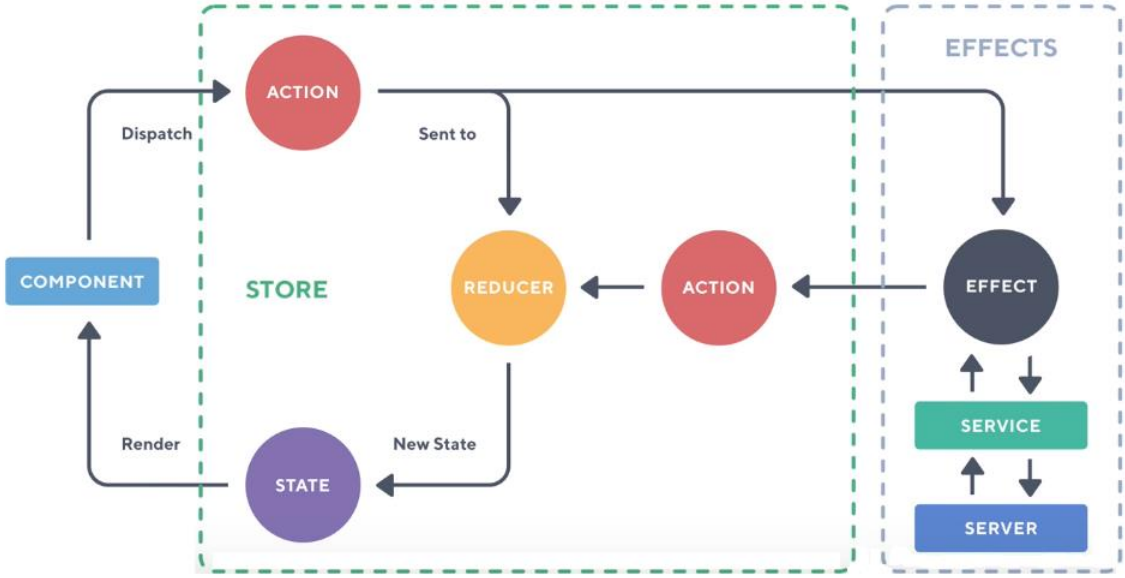
[۱] Redux

[۲] Store

[۳] Actions

[۴] Reducers

[۵] Effects



شکل ۱۸ - الگوی ریداکس[۶]

۴-۲-۴- ارتباط با سرور

ارتباط با سرور در انگولار به منظور تبادل داده‌ها با سرورها و انجام درخواست‌های اچ‌تی‌تی‌پی صورت می‌گیرد. این عملیات مهم برای بسیاری از برنامه‌های وب‌اپلیکیشن‌ها ضروری است. در ادامه، توضیحی در مورد ابزارها و اجزا اصلی ارتباط با سرور در انگولار ارائه می‌شود.

۱. **ماژول اچ‌تی‌تی‌پی کلاینت**[۱] در انگولار، برای استفاده از ارتباط با سرور از ماژول اچ‌تی‌تی‌پی کلاینت استفاده می‌شود. این ماژول باید به ماژول اصلی برنامه وارد شود. این ماژول فراهم‌کننده اچ‌تی‌تی‌پی کلاینت را در اختیار توسعه‌دهندگان قرار می‌دهد.

۲.**ایجاد سرویس برای ارتباط**: برای انجام عملیات ارتباط با سرور، می‌توان یک سرویس ایجاد کرد که از اچ‌تی‌تی‌پی کلاینت به عنوان وابستگی استفاده می‌کند. این سرویس معمولاً دارای رفتارهایی[۲] برای انجام عملیات مختلف مانند درخواست دریافت یا ارسال است.

---

[۱] HttpClientModule

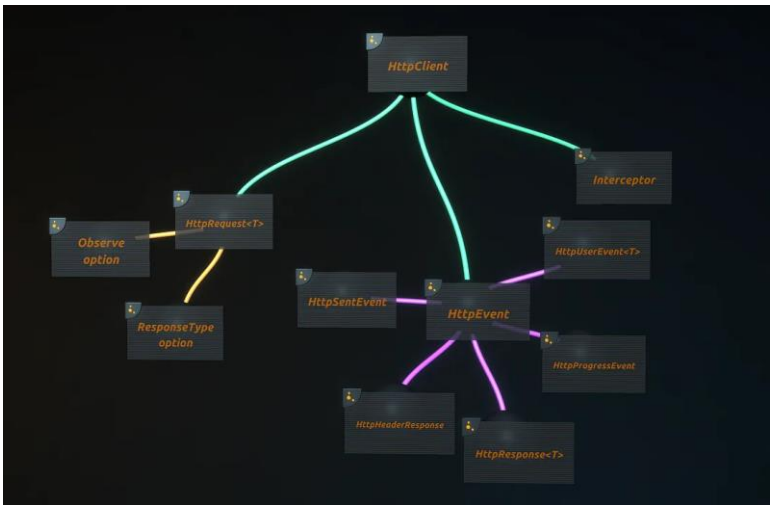
[۲] Methods

۳. **رهگیرها**[1]: رهگیرها در انگولار نقش مهمی در مدیریت و تغییر درخواست‌های اچ‌تی‌تی‌پی و پاسخ‌های سرور ایفا می‌کنند. این اجزای میانی بین درخواست و پاسخ اچ‌تی‌تی‌پی قرار می‌گیرند و به توسعه‌دهندگان این امکان را می‌دهند تا درخواست‌ها و پاسخ‌ها را تغییر دهند، خصوصیات جدید اضافه کنند و تغییراتی بر روی آنها اعمال کنند. از وظایف معمول رهگیرها می‌توان به موارد زیر اشاره کرد.

- **تغییر درخواست‌ها**: امکان افزودن هدرها، پارامترها، یا داده‌های اضافی به درخواست‌های اچ‌تی‌تی‌پی را فراهم می‌کنند.
- **پردازش پاسخ‌ها**: به توسعه‌دهندگان اجازه می‌دهند تا پاسخ‌های سرور را پیش‌پردازش کنند و اطلاعات مهم از پاسخ‌ها را استخراج کنند.
- **مدیریت خطاها**: به توسعه‌دهندگان امکان مدیریت خطاها را در درخواست‌های اچ‌تی‌تی‌پی می‌دهند. آنها می‌توانند خطاها را شناسایی کرده و اقدامات مناسبی را برای مدیریت خطاها انجام دهند.
- **اضافه کردن توکن‌های امنیتی**: می‌توانند به درخواست‌ها توکن‌های امنیتی مانند توکن‌های جی‌دبلیوتی[2]اضافه کنند تا امنیت درخواست‌ها تضمین شود. در شکل ۱۹ نحوه کارکرد کلی اچ‌تی‌تی‌پی کلاینت قابل مشاهده است.

[1] Interceptors

[2] JSON Web Token (JWT)



شکل ۱۹ - کارکرد اچ‌تی‌تی‌پی کلاینت[۷]

۴-۲-۴- پیاده‌سازی بک‌اند

جهت پیاده‌سازی بک‌اند ابتدا یک پروژه‌ی وب‌اِی‌پی‌آی جدید ایجاد کردیم. معماری پروژه‌های وب‌اِی‌پی‌آی در دات‌نت به این صورت است که دارای یک یا چند کنترل‌کننده[۱]است که وظیفه‌ی آن‌ها تعیین نقاط انتهایی است و رفتارها و ویژگی‌های لازم برای مدیریت یک درخواست اچ‌تی‌تی‌پی را دارد. مدل‌ها در پوشه‌ی موجودیت‌ها تعریف شده‌اند. برای ساخت مدل‌ها در پایگاه‌داده از رویکرد کدابتدا[۲] استفاده شده‌است. برای ارتباط با پایگاه داده از چارچوب موجودیت[۳]استفاده شده‌است. برای احراز هویت و مدیریت خطاها میان‌افزارها توسعه داده شده‌است.

میان‌افزار: میان‌افزارها در دات‌نت یک جزء بسیار مهم هستند که به توسعه‌دهندگان این امکان را می‌دهند تا فرآیند پردازش درخواست‌های وب را به صورت مرحله به مرحله تنظیم کنند. هر میان‌افزار در این خط پردازش وظیفه‌ای خاص دارد و می‌تواند عملیات‌های سفارشی خود را انجام دهد. از میان‌افزارها برای امور متنوعی استفاده می‌شود از جمله احراز هویت کاربران، توزیع درخواست‌ها به کنترل‌کننده‌های مختلف، مدیریت خطاها، ثبت لاگ‌ها و ارائه‌ی فایل‌های ایستا. همچنین، توسعه‌دهندگان می‌توانند

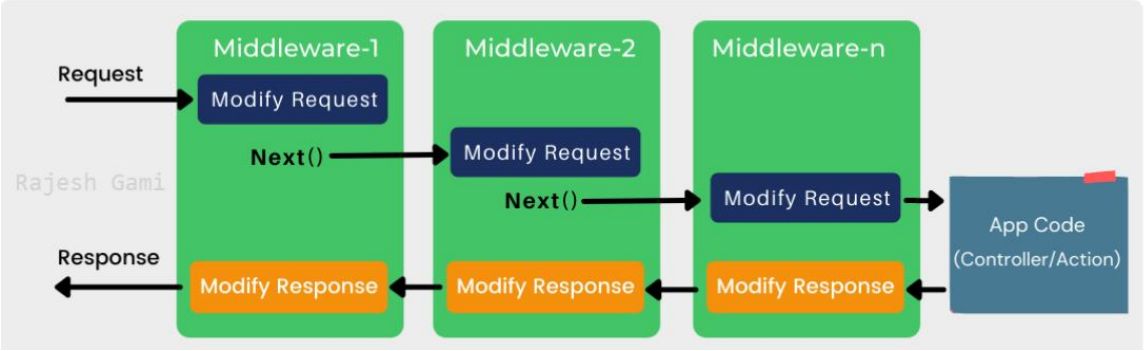
---

[۱] Controller

[۲] Code first

[۳] Entity Framework

میان‌افزارهای سفارشی خود را ایجاد و به فرآیند پردازش اضافه کنند. این اجزاء اساسی در طراحی و ایجاد برنامه‌های‌وب در دات‌نت، برای مدیریت درخواست‌ها و پاسخ‌ها و اجرای عملیات‌های مختلف در فرایند پردازش، بسیار مفید و انعطاف‌پذیر هستند.



شکل ۲۰ - کارکرد میان‌افزارها**[۸]**

چارچوب موجودیت: چارچوب موجودیت یک اوآرِام[۱]است. اوآرِام‌ها چارچوب‌هایی هستند که توسعه‌دهندگان را قادر می‌سازند تا به صورت شیءگرا با مدل‌های داده کار کنند و به صورت خودکار داده‌های ایجاد شده در کد را به ساختارهای مخصوص پایگاه داده مانند جداول و روابط تبدیل کنند. این چارچوب به عنوان یک او آر ام در دات‌نت استفاده می‌شود. از توابع لینک[۲]برای پرس‌وجوی داده‌ها استفاده می‌شود و برای تعریف مدل‌ها و پایگاه داده‌ها از رویکرد کد ابتدا استفاده می‌شود. توابع لینک این امکان را فراهم می‌کنند تا توسعه‌دهندگان به سادگی با مدل‌های شیءگرا کار کنند و نیازی به نوشتن دستورات اس‌کیو ال نداشته باشند، و این بسیاری از جزئیات پایگاه داده را از توسعه‌دهندگان مخفی می‌کند.

رویکرد کدابتدا: در این رویکرد ابتدا مدل‌های داده‌ای با استفاده از کلاس‌ها و ویژگی‌های دات‌نت تعریف می‌شود و سپس از چارچوب موجودیت برای ایجاد و مدیریت پایگاه داده‌ها استفاده می‌شود. این رویکرد این امکان را می‌دهد که به سادگی با مدل‌های داده‌ای شیءگرا کار شود و تغییرات در مدل‌ها را

[۱] Object-Relational Mapping

[۲] linq

به پایگاه داده منعکس کنند. پس از ایجاد مدل‌ها با استفاده از دستور مهاجرت[1] جدول‌ها ساخته می‌شوند.

```
src > housingCooperative > Domains > EntityConfigurations > C# LandProjectEntityConfiguration.cs > LandProjectEntityConfiguration > Configure
You, 2 months ago | 1 author (You)
using housingCooperative.Domains.Entities;

namespace housingCooperative.Domains.EntityConfigurations
{
    1 reference | You, 2 months ago | 1 author (You)
    public class LandProjectEntityConfiguration : IEntityTypeConfiguration<LandProjectEntity>
    {
        0 references
        public LandProjectEntityConfiguration()
        {
        }

        0 references
        public void Configure(Microsoft.EntityFrameworkCore.Metadata.Builders.EntityTypeBuilder<LandProjectEntity> builder)
        {


            builder.HasKey(x => x.Id);
            builder.Ignore(x => x._Id);
            builder.Property(x => x.CreatedAt).HasDefaultValueSql("getdate()");
            builder.Property(x => x.IsDeleted).HasDefaultValue(false);        You, 2 months ago • Entities,Configurations and entity enums ad
            builder.Property(x => x.IsVisibled).HasDefaultValue(true);
            builder.Ignore(x => x.ImageList);
            builder.HasMany(c => c.Phases).WithOne(c => c.LandProject).HasForeignKey(c => c.ProjectId).OnDelete(DeleteBehavior.NoAction);
            builder.HasMany(c => c.Plots).WithOne(c => c.LandProject).HasForeignKey(c => c.ProjectId).OnDelete(DeleteBehavior.NoAction);
            builder.ToTable($"tbl{nameof(LandProjectEntity)}");


        }
    }
}
```

شکل ۲۱ - رویکرد کدابتدا برای جدول پروژه

مهاجرت: مهاجرت در دات‌نت به عنوان یک ابزار اساسی برای مدیریت تغییرات در مدل‌های داده و پایگاه داده‌ها استفاده می‌شود. این فرایند به توسعه‌دهندگان این امکان را می‌دهد که تغییرات در مدل‌های داده را با استفاده از مهاجرت‌ها ایجاد و به پایگاه داده اعمال کنند. مهاجرت‌ها شامل تغییراتی همچون ایجاد جداول جدید، افزودن یا تغییر اجزا و ساختارهای پایگاه داده می‌شوند. این فرآیند به طور همگام تاریخچه تغییرات در پایگاه داده را نیز ذخیره می‌کند. با همگام‌سازی مدل‌های داده و پایگاه داده، توسعه‌دهندگان می‌توانند به راحتی تغییرات در مدل‌ها را به پایگاه داده انتقال داده و از هماهنگی بین این دو محیط اطمینان حاصل کنند.

[1] migration

```
src > housingCooperative > Migrations > C# 20240110062711_Description_To_Customer.cs > ...
1   using Microsoft.EntityFrameworkCore.Migrations;
2
3   #nullable disable
4
5   namespace housingCooperative.Migrations
6   {
        1 reference
7       public partial class Description_To_Customer : Migration
8       {
            0 references
9           protected override void Up(MigrationBuilder migrationBuilder)
10          {
11              migrationBuilder.AddColumn<string>(
12                  name: "Description",
13                  table: "tblCustomerEntity",
14                  type: "nvarchar(max)",
15                  nullable: true);
16          }
17
            0 references
18          protected override void Down(MigrationBuilder migrationBuilder)
19          {
20              migrationBuilder.DropColumn(
21                  name: "Description",
22                  table: "tblCustomerEntity");
23          }
24      }
25  }
26
```

شکل ۲۲ – فایل مهاجرت برای ایجاد توضیحات در جدول کاربر

۴-۲-۵- نقاط انتهایی

همانطور که ذکر شد نقاط انتهایی به عنوان واسط بین فرانت‌اند و بک‌اند نقش بسیار مهمی در کارکرد سامانه ایفا می‌کنند. جزییات نقاط انتهایی در جدول ۳ آمده است.

جدول ۳ - جزییات نقاط انتهایی

نام	نوع	وظیفه
ویرایش اطلاعات کاربر	PUT	با گرفتن داده‌های جدید، اطلاعات قدیمی کاربر را با اطلاعات جدید جایگزین می‌کند و روی اطلاعات کاربر موجود به‌روزرسانی انجام می‌دهد.
دریافت اطلاعات	GET	آی دی کاربر را دریافت کرده، و اطلاعات کاربر مورد نظر را

کاربر		برمی‌گرداند.
دریافت تمام پروژه‌های موجود	POST	پروژه‌های موجود را برمی‌گرداند.
دریافت تمام پروژه‌هایی که کاربر در آن‌ها مشارکت دارد	POST	با ورودی گرفتن آی دی فرم، جزییات فرم برگردانده می‌شود.
دریافت جزئیات یک پروژه	GET	آی دی پروژه را ورودی گرفته، پروژه به همراه واحدهای موجود و فازهای پروژه را به صورت تجمیع شده[1]بر می‌گرداند.
ایجاد قرارداد	POST	با دریافت اطلاعات واحد و کاربر قرارداد را برای کاربر مورد نظر ثبت می‌کند.
دریافت تمام قراردادهای کاربر	POST	با توجه به آی دی کاربر تمام قراردادهای کاربر را برمی‌گرداند.
دریافت جزئیات یک قرارداد	GET	با دریافت آی دی قرارداد، جزئیات قرارداد مورد نظر را برمی‌گرداند.
پرداخت اقساط یک قرارداد	PUT	با دریافت لیستی از آیدی‌های اقساط قرارداد و مبلغ قابل پرداخت کاربر، وضعیت اقساط مورد نظر کاربر به حالت پرداخت‌شده تغییر می‌کند.
دریافت تمام پرداخت‌های یک	POST	با توجه به آی دی کاربر، تمام پرداخت‌های کاربر را نمایش می‌دهد.

[1] Aggregated

کاربر		
دریافت جزئیات پرداخت یک کاربر	GET	با دریافت آی دی پرداخت، جزئیات آن پرداخت را نمایش می‌دهد.
دریافت اطلاعات تمام مشارکت‌کنندگان	POST	اطلاعات تمام کاربران سامانه را برمی‌گرداند.
دریافت تمام پرداخت‌های سامانه	POST	اطلاعات تمام پرداخت‌های سامانه را برمی‌گرداند.
ایجاد پروژه جدید	POST	با گرفتن اطلاعات پروژه، واحدها و فازها به صورت تجمیع شده، پروژه مورد نظر را به همراه واحدها و فازهای آن ثبت می‌کند.
ورود کاربر	POST	با ورودی گرفتن نام کاربری و رمز یک‌بار مصرف توکن صادر می‌شود.

۴-۳- تست نیازمندی‌ها

۴-۳-۱- نیازمندی‌های کارکردی

برای نقاط انتهایی تست واحد نوشته شد که تمامی آن‌ها با موفقیت آزموده شد. همچنین تمامی روندهای فرانت‌اند نیز توسط نرم‌افزار سلنیوم آزموده شد[۹].

```
Starting test execution, please wait...
A total of 1 test files matched the specified pattern.

Passed!  - Failed:     0, Passed:    28, Skipped:     0, Total:    28, Duration: 1 s - IntegerationWAF.dll (net6.0)
```

شکل ۲۳ – تست نقاط انتهایی

۴-۳-۲- نیازمندی‌های غیرکارکردی

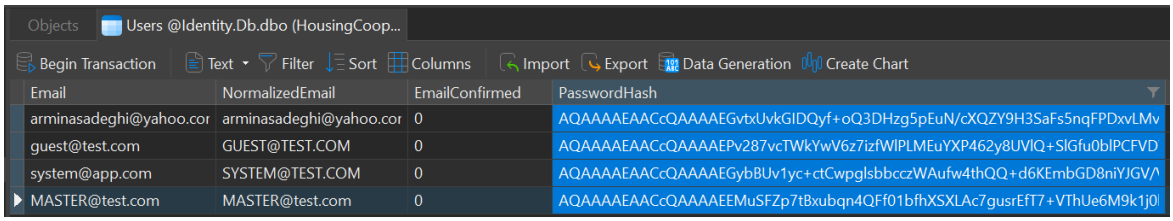
- **برای حفظ امنیت، مدیریت هویت و دسترسی در سامانه وجود داشته باشد.**

برای این نیازمندی کاربران برای ورود باید از نام کاربری و رمز عبو یک‌بارمصرف استفاده کنند. همچنین دو نقش کاربر و مدیر وجود دارد که دسترسی‌های هر کدام متفاوت از دیگری می‌باشد.

شکل ۲۲ – ورود به سامانه با رمز یکبار مصرف

- **محرمانگی داده‌های مهم حفظ شود.**

برای این نیازمندی داده‌های مهم و محرمانه کاربر به صورت هش[1]شده ذخیره می‌شود.



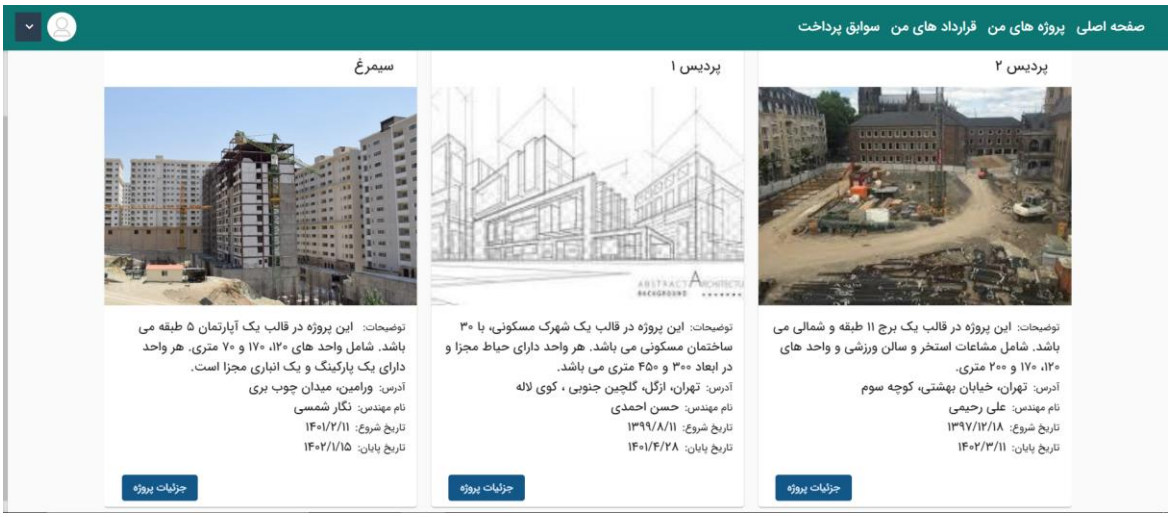
شکل ۲۵ – رمزهای هش‌شده در پایگاه داده

- **سامانه طراحی رابط کاربری متناسب داشته باشد.**

برای این نیازمندی طراحی و پیاده‌سازی مبتنی الگوی ترتیبی را درنظر گرفتیم تا در تمامی دستگاه‌ها، فارغ از سیستم‌عامل، قابل اجرا و به‌روزرسانی باشد.

---

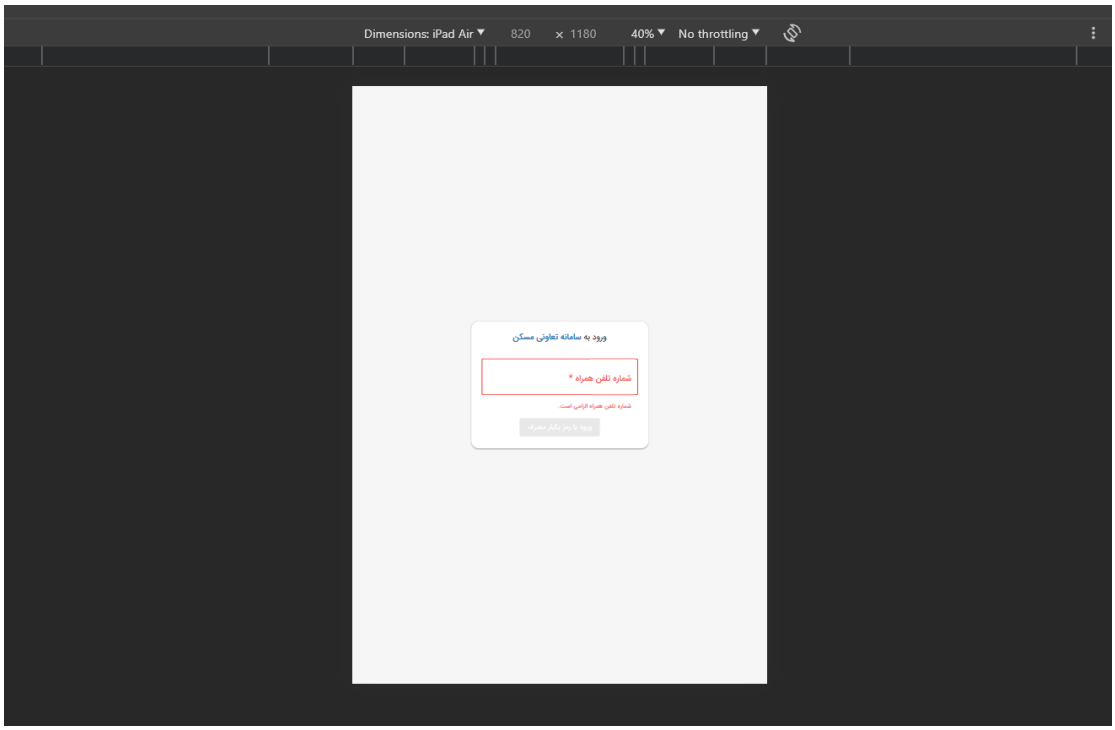
[1] Hash



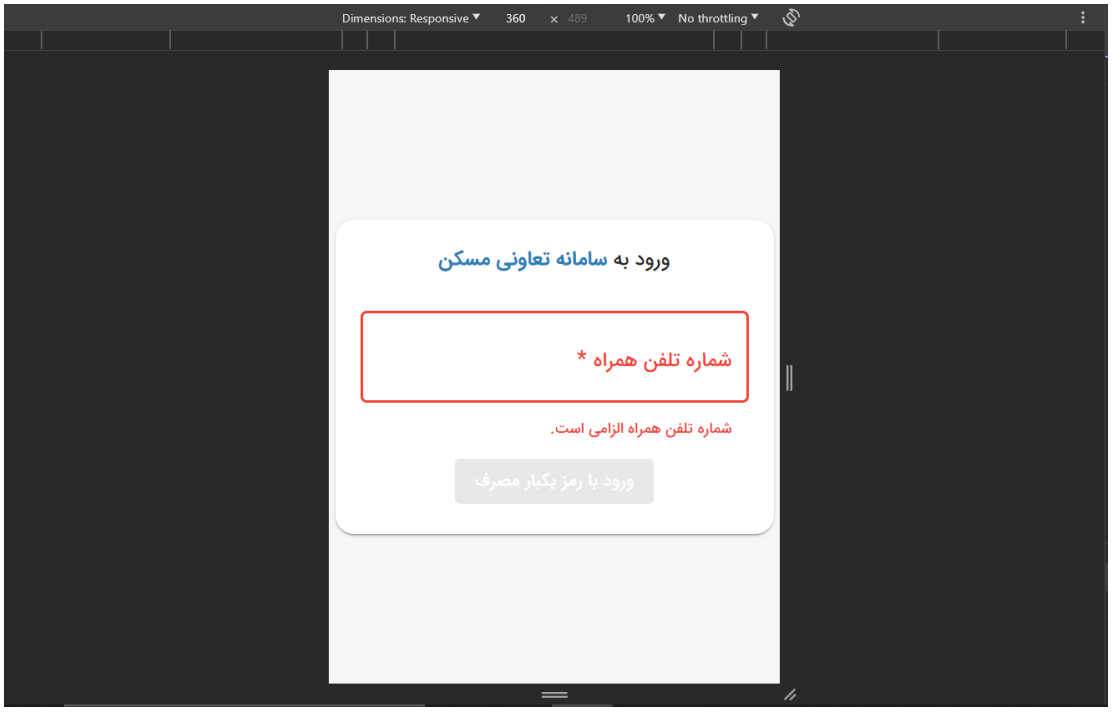
شکل ۲۶ – صفحه فرود پنل کاربری

- **طراحی سامانه باید در هر اندازه‌ی صفحه نمایش، اطلاعات لازم را به کاربر نمایش دهد.**

همانطور که اشاره کردیم رویکرد اولیه در طراحی و پیاده‌سازی، واکنش‌گرا بودن سامانه بوده است که در بخش طراحی آن را در فایل مربوط به فیگما طراحی کرده و در بخش چهارم آن را پیاده نمودیم. نمونه‌ی صفحه‌ی ورود در سه اندازه‌ی تبلت، موبایل و دسکتاپ قابل مشاهده است.



شکل ۲۷ - نمای صفحه‌ی ورود در تبلت

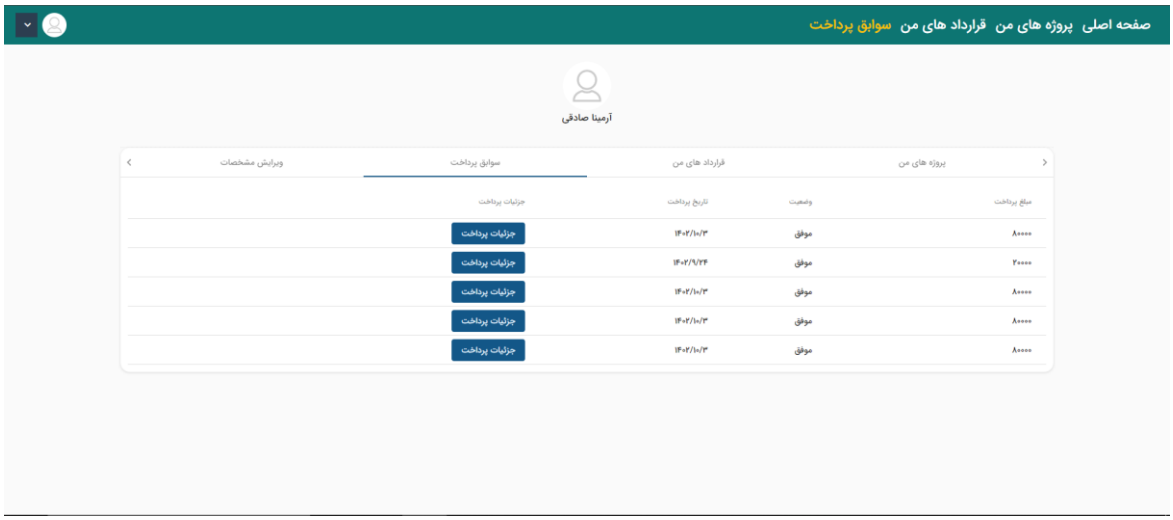


شکل ۲۸ - نمای صفحه‌ی ورود در موبایل

شکل ۲۹ - نمای صفحه‌ی ورود در دسکتاپ

- **سامانه باید ناوبری ساده و منطقی داشته باشد.**

در این پروژه از انتخاب به‌جای رنگ‌ها و تضاد رنگ‌های مناسب، اندازه‌ی فونت درست و قرارگیری به‌جای عناصر در جهت ساده کردن هرچه بیشتر ناوبری و استفاده از سامانه، بهره‌گیری شده‌است.



شکل ۳۰ - نمای صفحه‌ی ورود پرداخت‌های

- **برای حفظ کارایی سرعت بارگذاری کم باشد.**

در این سامانه سرعت بارگذاری هریک از صفحات کمتر از یک ثانیه می‌باشد.

۴-۴- جمع بندی

در این بخش ابتدا به توضیح نحوه‌ی پیاده‌سازی فرانت‌اند و بک‌اند پرداخته شد. سپس نیازمندی‌های مطرح شده در بخش سه مورد بررسی و ارزیابی قرار گرفت. تست نیازمندی‌های کارکردی با استفاده از عکس صفحات ارایه شد و در بخش نیازمندی‌های غیرکارکردی توضیحات مربوطه شرح داده شد.

فصل پنجم

جمع‌بندی و پیشنهادات

۵-۱- خلاصه

در این پژوهش هدف، طراحی و ایجاد یک سامانه‌ی مدیریت تعاونی‌های مسکن بود. در ابتدا برخی نمونه‌های مرتبط در این زمینه مشاهده و بررسی شد. سپس ایرادات و نقاط ضعف آن‌ها نسبت به شاخص‌های یک سامانه‌ی مدیریت تعاونی مسکن ایده‌آل شناسایی شد. سپس ویژگی‌های سامانه مشخص گردید و برای دستیابی به آن، نیازمندی‌های سامانه در دو بخش کارکردی و غیرکارکردی تعیین شد. بعد از گردآوری نیازمندی‌ها، فناوری سامانه با توجه به ملاک‌های بیان شده انتخاب شد و سامانه در سه بخش واسط کاربری، فرانت‌اند و بک‌اند طراحی شد و در انتها سامانه پیاده سازی شد و مورد آزمون قرار گرفت.

۵-۲- کاربرد

سازمان‌ها برای بهبود عملکردشان، احتیاج به ارتقاء سیستم‌های مدیریتی خود در طول زمان دارند. در این پروژه تلاش شد سامانه ای الکترونیکی به منظور بهبود عملکرد تعاونی‌های مسکن پیاده‌سازی شود. هدف از پیاده سازی این سامانه، ایجاد سامانه‌ای جامع، کاربردی، قابل توسعه و مقیاس پذیر برای کمک به هرچه بهتر شدن و بهینه‌تر شدن این سامانه‌ها بود. در این پروژه ویژگی‌هایی قرارداده‌شد که باعث افزایش بهره‌وری، سادگی مدیریت و شفافیت مالی در مدیریت تعاونی‌ها است. از این سامانه می‌توان در تعاونی‌های مسکن در جهت مدیریت پروژه‌ها، کاربران و پرداخت‌ها استفاده کرد.

۵-۳- پیشنهادات

پیشنهاد می‌شود برای افزایش کارایی این سامانه، امکانی ایجاد شود که به صورت ماهیانه، به کاربران جهت یادآوری پرداخت اقساطشان پیامکی ارسال شود. این ویژگی باعث تعامل هرچه بیشتر کاربر با سامانه می‌شود. همچنین می‌توان در کنار پروژه‌های مسکونی، مدیریت پروژه‌های فروش قطعه‌های زمین را نیز به این سامانه اضافه کرد که باعث گسترش فعالیت‌های تعاونی‌ها و مشارکت بیشتر کاربران می‌شود. علاوه بر این، میتوان با افزودن قابلیت در نظر گرفتن پاداش و جریمه در سامانه، کاربران را ترقیب به پرداخت منظم و به موقع اقساطشان کرد.

منابع و مراجع

[1] Angular (2023, December 17). Understanding Angular Overview [On-line]. Available: https://angular.io/guide/understanding-angular-overview.

[2] efcore image (2023, December 10). [On-line]. Available: https://www.learmoreseekmore.com/2022/11/dotnet7-api-crud-efcore.html

[3] Angular (2023, December 11). Understanding communicating with http [On-line]. Available: h https://angular.io/guide/understanding-communicating-with-http.

[4] Angular (2023, October 10). [On-line]. Available:https://angularindepth.com

[5] HTTP Client Angular (2023, December 10). [On-line]. Available https://medium.com/wevalue-expertise/angular-re-explained-1-httpclient-cdb2defbacd3

[6] Redux Pattern (2023, December 10). [On-line]. Available https://github.com/jmbl1685/redux-pattern-angular-react

[7] Angular (2023, December 10). [On-line]. Available https://thomasbroussard.fr/work/java/tutorials/advanced/angular/tutorial/basic-angular-tutorial.html

[8] Middleware (2023, December 10). [On-line]. Available https://www.shekhali.com/aspnet-core-middleware

[9] selenium (2023, December 10). [On-line]. Available https://www.selenium.dev